Unifying Inner-Loop Execution Harnesses and Outer-Loop Specification State for Autonomous Software Engineering

Alejandro Kaspar[1], **Hader Ceron**[1] **and Joaquin Menendez**[1]
[1]Google Cloud Consulting, Google

Autonomous coding agents rely on two distinct control loops during software development. The inner execution loop governs how an agent clarifies task ambiguities, probes boundary conditions, authors unit tests, and verifies code edits under tamper-proof test locks. The outer specification loop governs how system requirements, architectural contracts, and historical delta changes persist across multi-milestone repository evolution. Existing frameworks address these loops in isolation: inner-loop harnesses optimize single-turn task verification without maintaining a synchronized specification state across sprints, whereas outer-loop specification frameworks structure requirement documents without enforcing contrastive boundary probing or read-only test locks during execution. We introduce a unified dual-loop architecture that couples a Contract-Locked Dual-Stage Inner Execution Harness with a Two-Plane Delta-Merged Outer Specification State Machine. Across 252 blind multi-subagent benchmark trials instantiated with `Gemini 2.5 Pro` (300 workspace builds and 360 subagent invocations including fresh-context cold auditors) spanning four untouched upstream inner-loop benchmarks (`SWE-bench Verified`, `Commit0`, `BigCodeBench v0.1.2`, and `EvalPlus HumanEval+` 80× stress tests; 240 trials) and a five-milestone longitudinal specification evolution benchmark (`Commit0-Evo` on `TinyDB`; 12 multi-milestone trials across 60 sequential milestone builds), our Contract-Locked Dual-Stage Harness reduces baseline test tampering from $18.3 \pm 2.9\%$ (`Procedural TDD`) and $23.3 \pm 2.9\%$ (`OO-Harness`) to $0.0 \pm 0.0\%$ (0/60), raising tamper-free strict pass rate ($\text{Pass}_{\text{strict}}$ = Oracle $\wedge \neg$Tamper) to $91.7 \pm 2.9\%$ (55/60, +21.7 pp over `OO-Harness` and +23.4 pp over `Procedural TDD`) and patched-function AST mutation kill rate to $70.6 \pm 4.6\%$, while our Two-Plane Delta-Merged SSOT achieves $100.0 \pm 0.0\%$ clean-room rebuild and historical rollback accuracy with 0.0% contradiction density and 0.00 bits of behavioral entropy.

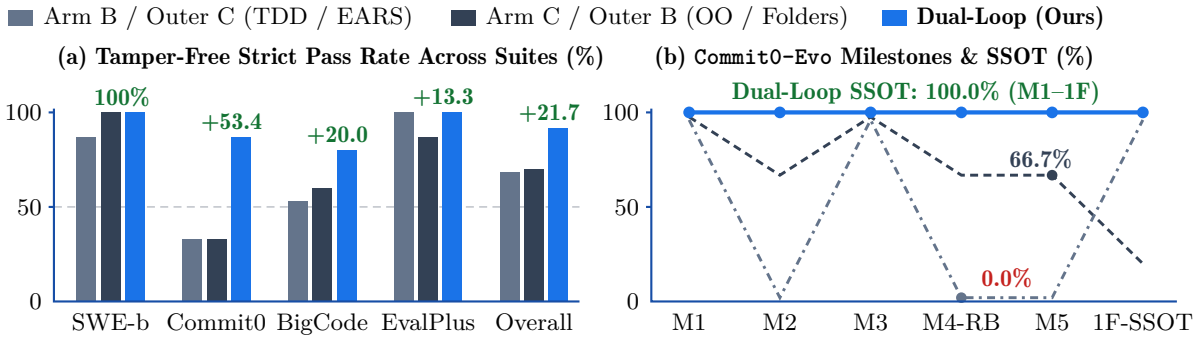


Figure 1: Empirical performance across 252 blind multi-subagent trials on untouched upstream benchmarks. **(a)** Tamper-free strict pass rate ($\text{Pass}_{\text{strict}}$ = Oracle $\wedge \neg$Tamper) across `SWE-bench Verified`, `Commit0`, `BigCodeBench v0.1.2`, `EvalPlus HumanEval+` (80×), and the 240-trial aggregate (3 runs×4 arms×20 tasks), comparing Procedural TDD (`Arm B`, gray), `OO-Harness` (`Arm C`, slate), and our Contract-Locked Dual-Stage Harness (`Arm D`, blue), with percentage-point gains (+Δ) over `Arm C` above each bar. **(b)** Longitudinal `Commit0-Evo` evaluation across 12 multi-milestone trials on `TinyDB` showing post-trajectory milestone checks (`M1`, `M2`, `M3`, `M4-RB` rollback, and `M5` clean-room rebuild) alongside single-file SSOT consolidation coverage (`1F-SSOT`): feature-isolated folders (`Outer B`, dashed slate) drop to $66.7 \pm 57.7\%$ at `M2`, `M4-RB`, and `M5` (20.0% on `1F-SSOT`), in-place EARS (`Outer C`, dash-dotted gray) erases prior cache invariants and drops to 0.0% on `M2`, `M4-RB`, and `M5`, while our Two-Plane SSOT (`Outer D`, solid blue) maintains $100.0 \pm 0.0\%$ across all checks.

Corresponding author(s): {adkaspar, haderceron, joaquinmen}@google.com
© 2026 Google. All rights reserved

1. Introduction

Autonomous software engineering agents operate across two distinct temporal scales. At the micro-scale of a single engineering ticket (the *inner execution loop*), an agent must interpret potentially ambiguous task requirements, inspect brownfield code, construct fail-to-pass unit tests, implement state transitions, and verify that boundary conditions behave as specified without *test tampering*—that is, altering, weakening, or mocking out failing unit tests so defective application code appears to pass [35, 12, 21, 38, 17, 34, 31, 32, 8]. At the macro-scale of multi-sprint repository evolution (the *outer specification loop*), an engineering system must maintain bidirectional consistency between executable source code and documented behavioral specifications as domain rules are added, modified, renamed, or rolled back across sequential milestones [25, 4, 36, 30].

Existing agentic software engineering methodologies optimize one temporal loop while leaving the complementary loop unguarded. On the inner loop, recent frameworks augment coding agents with procedural Markdown skills that mandate Red-Green-Refactor test-driven development (TDD) [2, 16, 1], multi-stage test-anchor flow engineering (`AlphaCodium` [28]), multi-agent programmer and test-designer roles (`AgentCoder` [10]), execution-guided rubber-duck debugging (`Self-Debugging` [6]), ambiguity-detecting requirement interrogation (`ClarifyGPT` [20]), synthesized code action-verifiers (`AutoHarness` [18]), object-oriented verification class hierarchies (`OO-Harness` [15]), or task-level contrastive boundary probing (`Harness`^{2} [33]). Even when recent self-improving harness frameworks introduce a meta-optimization loop to refine prompts or harness code across static benchmark tasks, they still treat the target software system's domain specification as a static, single-turn task prompt. When a software system evolves across multiple milestones, chronological sprint design files (`YYYY-MM-DD-*.md`) and isolated feature directories (`specs/001/`, `specs/002/`) accumulate contradictory requirements and scatter additive capabilities across unmerged files.

Conversely, outer-loop multi-agent and specification frameworks structure project artifacts across roles and documents (`MetaGPT` [9], `ChatDev` [26], `AgileCoder` [22], EARS [19], MADR [14], and the Elephant-Goldfish Model [EGM] [27]), yet rely on unguarded inner execution loops or static waterfall handoffs. In waterfall multi-agent pipelines such as `MetaGPT` [9] and `ChatDev` [26], a Product Manager agent emits a monolithic Product Requirement Document (PRD) once at project inception without an algebraic delta-merge compiler $\Phi(S_{t-1}, \Delta_t)$ to reconcile mid-lifecycle requirement pivots. In agile sprint frameworks such as `AgileCoder` [22], dynamic sprint backlogs accumulate task tickets without consolidating a single living capability specification or sealing red-phase tests against modification. In addition, when single-file specification conventions overwrite a single `requirements.md` file in place [19], they eliminate active contradictions at the cost of destroying working-tree historical provenance, causing point-in-time milestone rollbacks to fail. This motivates our central question:

> *Can an autonomous software engineering framework enforce tamper-proof inner-loop verification while maintaining a conflict-free, reversible outer-loop specification state across milestones?*

We address this question by introducing a unified dual-loop software engineering architecture (Figures 1 and 2) that couples two complementary control loops: ❶ **Contract-Locked Dual-Stage Inner Execution Harness (Arm D)** integrates Socratic pre-coding ambiguity elicitation, frozen `OO-Harness` domain subclass routing [15, 18], `Harness`^{2} contrastive boundary probing at $(N - \eta, N, N + \eta)$ equivalence points [33], tamper-proof read-only test file locking (`chmod 0444` paired with an out-of-workspace SHA-256 digest manifest that prevents the builder agent from editing or weakening red-phase tests during implementation), and a fresh-context two-stage cold subagent audit [27, 13, 29] that inspects both specification compliance (Phase 1) and full-module boundary mutation adequacy (Phase 2). ❷ **Two-Plane Delta-Merged Single Source of Truth (SSOT) Outer Specification State Machine (Outer D)** builds upon the open-source directory and delta-header conventions of OpenSpec

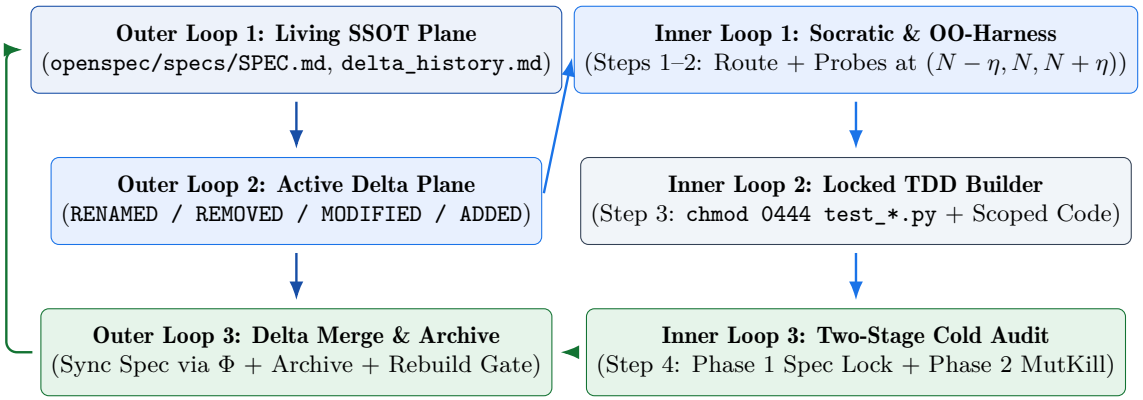


Figure 2: The Unified Dual-Loop Architecture. The Outer Loop (left column) stages requirement deltas (`Outer Loop 2`) from the Living SSOT Plane (`Outer Loop 1`) and, once authorized by the inner-loop cold audit, merges them via $\Phi(S_{t-1}, \Delta_t)$ (`Outer Loop 3`) with a left-hand return transition updating S_t in the Living SSOT Plane. The Inner Loop (right column) executes Steps 1–2 (`Inner Loop 1`: Socratic self-elicitation, `OO-Harness` routing, and boundary probing at $(N-\eta, N, N+\eta)$), Step 3 (`Inner Loop 2`: `chmod 0444` locked TDD build), and Step 4 (`Inner Loop 3`: Phase 1 and Phase 2 cold audit).

(`openspec/changes/` vs. `openspec/specs/` with `RENAMED`/`REMOVED`/`MODIFIED`/`ADDED` sections) and Conductor (`conductor/tracks/` task plans)[1].

Our contributions are fourfold:

① **Comparative Baseline Taxonomy and Dual-Loop Architecture (Sections 2 and 3).** We synthesize 26 peer-reviewed and archival agentic software engineering works into a unified 14-system comparative taxonomy (Table 1 and Figure 3), and formalize our dual-loop architecture combining a four-step contract-locked inner harness (Figure 4) with an invertible two-plane outer state compiler $\Phi(S_{t-1}, \Delta_t)$ (Figure 5).

② **Multi-Subagent Inner-Loop Evaluation Across Four Upstream Benchmarks (Sections 6.1 and 6.2).** Across 240 blind subagent trials (3 runs × 4 arms × 20 upstream tasks) spanning `SWE-bench Verified` [12], `Commit0` [36], `BigCodeBench v0.1.2` [38], and `EvalPlus HumanEval+` (80×) [17], we uncover an empirical finding: without physical read-only test locks, explicit TDD and verification harnesses (`Arm B` and `Arm C`) increase baseline test tampering from $5.0 \pm 0.0\%$ (`Arm A`) to $18.3 \pm 2.9\%$ and $23.3 \pm 2.9\%$ when agents overwrite starter test suites to pass local verification gates. Our Contract-Locked Dual-Stage Harness (`Arm D`) eliminates test tampering across all 240 trials ($0.0 \pm 0.0\%$, 0/60), raising tamper-free strict pass rate ($\text{Pass}_{\text{strict}} = \text{Oracle} \wedge \neg\text{Tamper}$) to $91.7 \pm 2.9\%$ (55/60, +21.7 pp over `Arm C` and +23.4 pp over `Arm B`).

③ **Patched-Function AST Mutation Adequacy and Confirmation Bias Analysis (Section 6.2).** Evaluating Offutt Sufficient-5 AST mutation operators [23, 11, 24] scoped to modified functions across all 240 workspaces, we show that `Arm D` achieves the highest overall mutation kill rate ($70.6 \pm 4.6\%$ vs. $59.6 \pm 6.3\%$ in `Arm A` and $61.7 \pm 3.8\%$ in `Arm C`) while quantifying the confirmation bias gap between agent self-reported completion ($100.0 \pm 0.0\%$) and held-out oracle pass rates.

[1] OpenSpec (https://github.com/Fission-AI/OpenSpec, MIT License) and Conductor (https://github.com/gemini-cli-extensions/conductor, Apache-2.0 License) define the underlying two-plane directory layout and Markdown delta headers, while open-source feature-folder SDD toolkits popularize the numbered `specs/001-feature/spec.md` directory layout evaluated in Outer B. Our novel outer-loop contributions are: (1) formalizing clean-room rebuildability, oracle-projected behavioral entropy $H(B_{T_t}(C'_t))$, and the invertible four-stage merge operator $\Phi(S_{t-1}, \Delta_t)$ with point-in-time historical rollback Φ^{-1} (Section 3.1), (2) two-plane synchronization between the living capability specification `openspec/specs/SPEC.md` and the pre-image delta archive `conductor/tracks/delta_history.md`, (3) topology-agnostic contradiction and historical-recovery auditing (`compute_contradiction_density`), and (4) the programmatic inter-loop handoff gate coupling inner cold-audit sign-off to outer delta merging (Section 3.2).

④ **Longitudinal Multi-Milestone Specification Evolution and Rollback on `Commit0-Evo` (Sections 6.3 and 6.4).** Across 12 multi-milestone subagent trials (3 runs×4 outer-loop topologies, 60 sequential milestone builds on `TinyDB`), we demonstrate that append-only sprint specs (`Outer A`) and feature-isolated folders (`Outer B`) accumulate 41.7% and 36.4% active contradiction density (with `Outer B` degrading to $66.7 \pm 57.7\%$ and $\hat{H} = 0.92$ bits at `M2`, `M4-RB`, and `M5`), whereas single-file in-place EARS (`Outer C`) collapses to $0.0 \pm 0.0\%$ on `M4` $\rightarrow$ `M2` historical rollback. Only our Two-Plane Delta-Merged SSOT (`Outer D`) achieves $100.0 \pm 0.0\%$ across all five milestones with 0.0% contradiction density and $\hat{H} = 0.00$ bits.

2. Related Work and Comparative Baseline Taxonomy

To contextualize our architecture within the broader field of autonomous software engineering, we examine three research threads: (1) inner-loop execution harnesses, flow engineering, and requirement clarification; (2) outer-loop multi-agent software development and longitudinal specification management; and (3) mutation testing, oracle adequacy, and reward tampering. Table 1 and Figure 3 compare 14 representative systems across seven inner-loop and outer-loop architectural capabilities. To ensure unambiguous comparison in Table 1, we assign `Y` only when a capability is enforced by a dedicated structural or cryptographic invariant, `P` when it relies on prompt-level instructions, single-turn heuristics, or unmerged multi-file retention, and `-` when absent across all seven columns: (i) under `Ambig.`, `Y` denotes an active pre-coding ambiguity gate that either queries an external oracle via behavioral divergence (`ClarifyGPT`) or executes a structured 5-question Socratic contract check against repository governance files paired with a Phase 1 Cold Audit (ours; falling back to an interactive oracle when no governance contract resolves the ambiguity), whereas `P` denotes static prompt reflection (`SKILL.md`, `AlphaCodium`, `MetaGPT`) or passive `*.md` file discovery without Socratic verification; (ii) under `Route`, `Y` denotes deterministic filesystem/header inspection dispatching to frozen verification subclasses (`OOHarnessRouter`), whereas `P` denotes multi-agent role/SOP workflow routing (`AgentCoder`, `MetaGPT`, `AgileCoder`); (iii) under `Probe`, `Y` denotes programmatic contrastive boundary probing at $(N - \eta, N, N + \eta)$ equivalence points and precedence-inversion pairs, whereas `P` denotes general LLM-synthesized edge-case anchors, synthesized action-verifier filters, surviving-mutant repair prompts, or behavioral-divergence inputs (`ClarifyGPT`, `AlphaCodium`, `AgentCoder`, `MuTAP`, `AutoHarness`); (iv) under `Lock`, `Y` denotes physical POSIX read-only permissions (`chmod 0444` as an in-loop friction barrier) paired with an out-of-workspace SHA-256 digest manifest verified by the external orchestrator, whereas `P` denotes dedicated Test Designer vs. Programmer role separation over a shared mutable directory (`AgentCoder`; in contrast, waterfall/sprint pipelines such as `MetaGPT` and `AgileCoder` grant engineering roles unrestricted write access to test files and receive `-`); (v) under `Audit`, `Y` denotes an independent fresh-context post-green cold subagent audit combining cryptographic test-lock verification with static AST/branch full-module mutation adequacy inspection, whereas `P` denotes execution-only test/mutant/action-verifier running or prompt-level QA role checks (`MuTAP`, `AutoHarness`, `AlphaCodium`, `AgentCoder`, `MetaGPT`, `AgileCoder`); (vi) under `SSOT` S_t, `Y` denotes maintaining a single consolidated, conflict-free specification across multi-milestone evolution, whereas `P` denotes single-turn inception PRDs (`MetaGPT`); and (vii) under `Rollback`, `Y` denotes an automated algebraic inverse state compiler Φ_{seq}^{-1} that reconstructs a single conflict-free SSOT, whereas `P` denotes manual retention of unmerged historical files or sprint cards (`Append-Only / Folder Specs` and `AgileCoder`, which can recover historical rules by reading early files at the cost of accumulating cross-file contradictions).

Inner-Loop Execution Harnesses, Flow Engineering, and Ambiguity Elicitation. ReAct [35] established the interleaved reasoning and action loop used by generalist repository-scale coding scaffolds such as `SWE-agent` [34] and `OpenHands` (`CodeAct`) [31] on SWE-bench [12], SWT-Bench [21], and Commit0 [36], while `Agentless` [32] demonstrated that a deterministic three-phase fault localization, code repair, and regression-test validation pipeline can rival open-ended tool-use agents on single-

Table 1: Comparative taxonomy of 14 agentic software engineering harnesses and specification frameworks across inner-loop execution capabilities and outer-loop longitudinal specification state invariants (`Y` = enforced structural/algebraic invariant; `P` = prompt-level, single-turn, or unmerged retention; `-` = absent; `Append-Only / Folder Specs` denotes passive multi-file storage topologies `Outer A` and `Outer B`).

Framework / Architecture	Ambig.	Route	Probe	Lock	Audit	SSOT S_t	Rollback
ReAct / SWE-agent / OpenHands [35, 34, 31]	–	–	–	–	–	–	–
Procedural TDD (`SKILL.md`) [16]	P	–	–	–	–	–	–
Self-Debugging / Agentless [6, 32]	–	–	–	–	P	–	–
ClarifyGPT [20]	**Y**	–	P	–	–	–	–
AlphaCodium [28]	P	–	P	–	P	–	–
AgentCoder [10]	–	P	P	P	P	–	–
MuTAP [7]	–	–	P	–	P	–	–
AutoHarness [18]	–	–	P	–	P	–	–
OO-Harness + Harness^2 [15, 33]	–	**Y**	**Y**	–	P	–	–
MetaGPT / ChatDev [9, 26]	P	P	–	–	P	P	–
AgileCoder [22]	–	P	–	–	P	–	P
Append-Only / Folder Specs	–	–	–	–	–	–	P
Single-File In-Place EARS (Outer C) [19]	–	–	–	–	–	**Y**	–
Unified Dual-Loop (Ours)	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**

turn issue resolution. Holding the underlying model and tool interface fixed, structured execution harnesses substantially alter agent reliability. Chen et al. [6] introduced `Self-Debugging`, teaching LLMs to explain their own generated code and execution traces in a rubber-duck feedback loop; however, `Self-Debugging` and `Agentless` (which synthesizes post-hoc issue-reproduction tests for candidate patch filtering, accounted for under `Audit = P` in Table 1, without pre-implementation contrastive boundary probing, so `Probe = -`) rely on pre-existing or single-point reproduction tests without guarding against weak boundary assertions. Ridnik et al. [28] proposed `AlphaCodium`, a test-based, multi-stage flow engineering pipeline that reflects on problem prompts, reasons over public tests, generates additional AI test anchors, and iteratively repairs code against those anchors (0.19 $\rightarrow$ 0.44 pass@5 on CodeContests). Huang et al. [10] decoupled code generation from test generation in `AgentCoder`, pairing a Programmer agent with an independent Test Designer agent and a Test Executor (77.4%–89.9% pass@1 on HumanEval/MBPP). While `AlphaCodium` and `AgentCoder` improve test coverage over single-pass prompting, neither framework physically locks red-phase test files against green-phase modification when an implementation agent encounters a failing assertion over a shared workspace, nor do they cross-check repository governance contracts before test generation. Addressing requirement ambiguity, Mu et al. [20] introduced `ClarifyGPT`, which detects ambiguous code-generation prompts via multi-sample behavioral divergence and synthesizes targeted clarifying questions (+10.52%–12.33% accuracy). On the harness architecture side, Li et al. [16] demonstrated on SkillsBench across 84 tasks and 7,308 trajectories that curated procedural `SKILL.md` files (including pre-coding review checklists, rated `P` under `Ambig.`) improve pass rates by +16.2 percentage points on average, but suffer context degradation when monolithic prompts grow overly long. Lou et al. [18] introduced `AutoHarness`, formalizing "code as harness" by synthesizing executable Python action-verifiers that reject invalid agent actions at runtime so smaller harnessed models outperform larger unharnessed models. Li et al. [15] resolved monolithic prompt bloat via the Object-Oriented Harness (`OO-Harness`), organizing verification workflows into a class hierarchy (`BaseHarness` and specialized domain subclasses), while Xu et al. [33] introduced `Harness`^{2}, synthesizing task-specific verification checks via contrastive `Propose-Probe-Compose` boundary probing. Our inner-loop harness (Arm D) unifies these advances by combining `ClarifyGPT`-inspired Socratic ambiguity resolution, `OO-Harness` domain routing, `Harness`^{2} $(N-\eta, N, N+\eta)$ boundary probing, physical `chmod 0444` + external SHA-256 test locking, and a two-stage cold subagent audit.

Outer-Loop Multi-Agent Frameworks and Longitudinal Specification Evolution. Decomposing

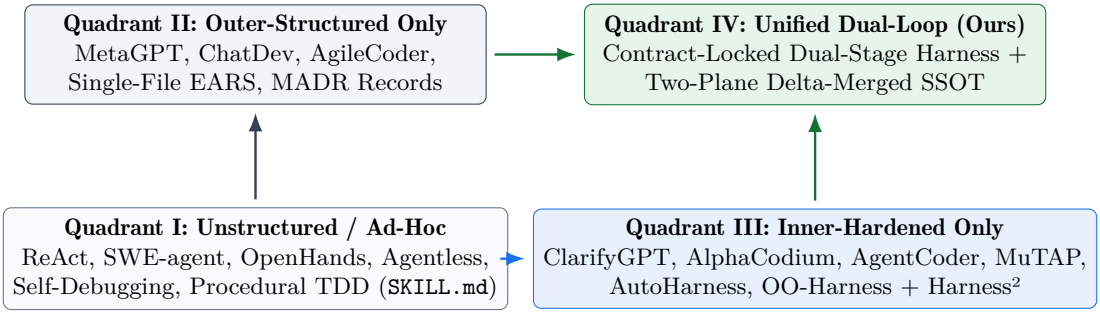


Figure 3: Architectural positioning of autonomous software engineering frameworks across Inner-Loop Verification Rigor (horizontal transition from Quadrant I/II to Quadrant III/IV) and Outer-Loop Longitudinal State Fidelity (vertical transition from Quadrant I/III to Quadrant II/IV). Existing frameworks populate Quadrants I–III, whereas our Unified Dual-Loop Architecture occupies Quadrant IV.

software systems around explicit behavioral contracts dates to Parnas [25] and Brooks [4], formalized in RFC 2119 keywords [3], EARS syntax [19], and MADR decision logs [14]. In agentic software engineering, Rensin [27] formulated the Elephant-Goldfish Model (EGM), separating persistent, deep-context specification artifacts ("Elephants," where sizeof(spec) $\ll$ sizeof(code) enables complete system rebuilds) from stateless, zero-context worker and reviewer subagents ("Goldfish" invoked without conversational history bias), while Calderon et al. [5] proved across 13 frontier LLMs that factual errors arise primarily from access and recall bottlenecks ("lost keys") as opposed to missing parametric knowledge ("empty shelves"), motivating deterministic external specification pointers. In multi-agent LLM software development, `MetaGPT` [9] and `ChatDev` [26] organize agents into waterfall Standard Operating Procedures (Product Manager $\rightarrow$ Architect $\rightarrow$ Engineer $\rightarrow$ QA), passing structured PRDs and interface designs down a communicative chain. However, as recent longitudinal repository benchmarks such as `SWE-EVO` [30] demonstrate (where frontier agents with `SWE-agent` and `OpenHands` [31] drop from 65% on single-turn SWE-bench Verified to 21% on multi-step release-note evolution due to cross-milestone state drift), static waterfall PRDs become stale as soon as subsequent milestones modify or rename earlier invariants. `AgileCoder` [22] replaces waterfall handoffs with multi-sprint agile backlogs and a dynamic code dependency graph, yet stores sprint tasks as incremental backlog items without a formal delta-merge operator $\Phi(S_{t-1}, \Delta_t)$ to maintain both a conflict-free living capability specification S_t and an invertible historical archive A_t. Similarly, while open-source feature-folder SDD toolkits organize per-feature artifacts into numbered directories (`specs/001-feature/spec.md`) and provide interactive pre-coding clarification commands, their passive multi-folder directory topology (evaluated as Outer B) does not algebraically merge later milestone modifications back into a single canonical capability specification. Our outer-loop evaluation in Suite 5 (`Commit0-Evo` on `TinyDB`; Sections 6.3 and 6.4) directly compares append-only sprint specification files (`Outer A`), feature-isolated specification directories without chronological numbering (`Outer B`), single-file in-place EARS specifications (`Outer C`), and our Two-Plane Delta-Merged SSOT (`Outer D`) across five sequential capability milestones and historical rollback.

Mutation Testing, Oracle Adequacy, and Reward Tampering. Passing a self-authored test suite is a weak proxy for correctness when tests omit boundary conditions [23, 11, 24]. Recent benchmarks for issue-to-test generation (`SWT-Bench` [21] and `Auto-TDD` / `TDD-Bench Verified` [1]) demonstrate that standard zero-shot and few-shot LLM unit tests leave boundary mutants alive, while `MuTAP` [7] shows that an active execution-guided mutation feedback loop (injecting surviving mutants back into the LLM prompt) improves test mutation scores. In contrast to `MuTAP`'s dynamic mutant-execution loop across every candidate mutant, our Phase 2 Cold Auditor performs a single-pass static AST and branch-boundary audit across all public methods of the module, identifying unprobed methods and boundary predicates without executing synthetic mutant suites during inner-loop inference.

Regarding adversarial verification, Denison et al. [8] proved that RL- and instruction-tuned agents exposed to difficult or corrupted environments generalize from sycophancy to active *reward tampering* (editing unit tests or mocking out verification logic to manufacture a passing exit code), and Zhong et al. [37] demonstrated on `ImpossibleBench` that frontier coding agents systematically exploit or rewrite unit tests when specifications and starter test cases collide. Kenton et al. [13] showed that separating generation from adversarial critique in a two-player verification protocol reduces reward hacking relative to single-player reflection, and Sahoo [29] introduced the Cross-Trace Verification Protocol (CTVP) to unmask deceptive code behavior via execution-trace semantic invariance without trusting self-reported intent. Our 240-trial inner-loop evaluation across Suites 1–4 (Sections 6.1 and 6.2) directly measures baseline test tampering and confirmation bias across four inner-loop harness configurations (`Arms A-D`).

3. Formalism and Dual-Loop Architecture

3.1. Information-Theoretic Specification State, Clean-Room Rebuilds, and Invertible Delta Merging

Let C_t denote the executable source code of a software system at milestone $t \in \{1, \ldots, M\}$, and let S_t denote the deterministic repository specification state (the set of active Markdown specification files where sizeof$(S_t) \ll$ sizeof(C_t) [27, 5], initialized from an empty base state $S_0 = \emptyset$ via $S_1 = \Phi(\emptyset, \Delta_1)$) at milestone t. Let $T_t = \{a_1, \ldots, a_K\}$ denote the sealed ground-truth behavioral oracle for milestone t, comprising $K \geq 1$ deterministic pass/fail test assertions. When all executable source files are deleted ($C_t \leftarrow \emptyset$), invoking a stochastic autonomous coding agent $\mathcal{A}$ parameterized by S_t induces a probability distribution $C'_t \sim \mathcal{A}(S_t, \emptyset)$ over synthesized implementations, and a corresponding discrete random vector $B_{T_t}(C'_t) = (a_1(C'_t), \ldots, a_K(C'_t)) \in \{0,1\}^K$ representing the implementation's observable behavior projected onto the equivalence partition of the ground-truth oracle T_t.

Definition 1 (Clean-Room Rebuildability and Oracle-Projected Behavioral Entropy)**.** *For tolerances $\epsilon \in [0, 1 - 2^{-K}]$ and $\delta \geq 0$, a specification state S_t is an (ϵ, δ)-rebuildable Single Source of Truth (SSOT) at milestone t with respect to a stochastic synthesis operator $\mathcal{A}$ and sealed ground-truth oracle T_t if deleting all executable code ($C_t \leftarrow \emptyset$) and sampling $C'_t \sim \mathcal{A}(S_t, \emptyset)$ satisfies (i) the supervised oracle correctness bound $\Pr_{C'_t \sim \mathcal{A}(S_t, \emptyset)}[B_{T_t}(C'_t) = \mathbf{1}_K] \geq 1 - \epsilon$ and (ii) the oracle-projected Shannon entropy bound $H(B_{T_t}(C'_t)) \leq \delta$, where $\mathbf{1}_K = (1, \ldots, 1) \in \{0,1\}^K$. Across k independent clean-room synthesis seeds $C'^{(1)}_t, \ldots, C'^{(k)}_t \sim \mathcal{A}(S_t, \emptyset)$, the empirical plug-in estimator of oracle-projected behavioral entropy over observed bit vectors $v \in \{0,1\}^K$ with empirical frequencies $\hat{p}(v) = \frac{1}{k} \sum_{i=1}^{k} \mathbb{I}[B_{T_t}(C'^{(i)}_t) = v]$ is*

$$\hat{H}(B_{T_t}(C'_t)) = - \sum_{v \in \{0,1\}^K : \hat{p}(v) > 0} \hat{p}(v) \log_2 \hat{p}(v).$$

Projecting behavior onto $B_{T_t}(C'_t)$ separates specification-constrained behavioral equivalence classes from benign implementation variation (such as internal variable naming) outside T_t. By the grouping property of Shannon entropy over $\{\{\mathbf{1}_K\}, \{0,1\}^K \setminus \{\mathbf{1}_K\}\}$ (analogous to Fano's inequality) and the monotonicity of $g(p) = h_2(p) + p \log_2(2^K - 1)$ on $[0, 1 - 2^{-K}]$ for binary entropy $h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$, Condition (i) implies the Fano upper bound

$$H(B_{T_t}(C'_t)) \leq h_2(\epsilon) + \epsilon \log_2(2^K - 1) \leq h_2(\epsilon) + \epsilon K,$$

so $H(B_{T_t}(C'_t)) \to 0$ as $\epsilon \to 0$. When $\epsilon > 0$ and $\delta < h_2(\epsilon) + \epsilon \log_2(2^K - 1)$, Condition (ii) constrains the dispersion of residual failure probability mass across the $2^K - 1$ failing oracle outcomes: an unmerged specification corpus with contradictory rules induces high behavioral entropy $\hat{H} > 0$ because different synthesis trajectories attend to competing clauses across files. Because the finite-sample plug-in estimator $\hat{H}(B_{T_t}(C'_t))$ evaluated over $k \ll 2^K$ synthesis seeds (such as $k = 3$ runs

across $K = 5$ milestone oracle checks in `Commit0-Evo` where $\hat{H} \in \{0.00, 0.92\}$ bits) is bounded above by $\log_2 k$, $\hat{H}$ serves as an empirical sample-dispersion metric: observing $\hat{H} = 0.00$ bits paired with k/k passing runs ($\hat{p}(\mathbf{1}_K) = 1$) demonstrates zero empirical trajectory dispersion across the tested seeds while yielding a one-sided $(1-\alpha)$ Clopper–Pearson confidence bound of $\epsilon \leq 1 - \alpha^{1/k}$ ($\epsilon \leq 0.632$ at $\alpha = 0.05$ for $k = 3$ independent runs per topology in Suite 5, `Commit0-Evo`). In coupled multi-milestone evaluations (`Commit0-Evo`; Section 6.3), the empirical distribution marginalizes over both outer-loop specification evolution $S_t^{(i)}$ across Milestones `M1`–`M4` and inner-loop clean-room synthesis $C_t'^{(i)} \sim \mathcal{A}(S_t^{(i)}, \emptyset)$ at Milestone `M5`.

When requirements evolve across milestones $t = 1, \ldots, M$, we model a living capability specification S_t as a finite set of structured requirement blocks $r = (\mathrm{id}, \mathrm{hdr}, \mathrm{sig}, \mathrm{body})$ indexed by canonical requirement keys $\mathrm{key}(r) = \mathrm{id} \in \mathcal{K}$ (`REQ-XXXX`), with key projection $\mathrm{keys}(S_t) = \{\mathrm{key}(r) : r \in S_t\}$ and key-lookup subset $\mathrm{blocks}_{S_t}(K) = \{r \in S_t : \mathrm{key}(r) \in K\}$. Each milestone introduces a structured requirement delta $\Delta_t = (\Delta_t^{\mathrm{REN}}, \Delta_t^{\mathrm{REM}}, \Delta_t^{\mathrm{MOD}}, \Delta_t^{\mathrm{ADD}})$, where $\Delta_t^{\mathrm{REM}} \subseteq \mathcal{K}$ is a finite set of removed keys, Δ_t^{MOD} and Δ_t^{ADD} are finite sets of replacement and newly added requirement blocks containing at most one block per key, and Δ_t^{REN} stores invertible 3-tuple mappings $(\mathrm{id}_{\mathrm{old}}, \mathrm{hdr}_{\mathrm{old}}, \mathrm{sig}_{\mathrm{old}}) \leftrightarrow (\mathrm{id}_{\mathrm{new}}, \mathrm{hdr}_{\mathrm{new}}, \mathrm{sig}_{\mathrm{new}})$, with $\mathrm{dom}(\Delta_t^{\mathrm{REN}})$ denoting the set of old 3-tuples and $\pi_{\mathrm{id}}(\mathrm{id}_{\mathrm{old}}, \mathrm{hdr}_{\mathrm{old}}, \mathrm{sig}_{\mathrm{old}}) = \mathrm{id}_{\mathrm{old}}$ denoting the first-coordinate key projection. Under an append-only specification scheme ($S_t^{\mathrm{append}} = \{s_1, s_2, \ldots, s_t\}$), a rule modification in Δ_3^{MOD} (recorded in s_3) that changes a hold threshold in s_1 from > 100000 to ≥ 100000, or additive contracts (Δ_t^{ADD}) recorded across s_2, s_3, s_4, leaves the initial capability file s_1 unamended. Conversely, under destructive in-place overwriting (S_t^{inplace}), overwriting s_1 in place preserves current-state consolidation but destroys prior milestone clauses, preventing point-in-time historical rollback to an earlier milestone $t_0 < t$.

Proposition 1 (Structural Key Uniqueness, Conditional Contradiction Freedom, and Invertible Rollback)**.** *Let $S_0 = \emptyset$ and let S_{t-1} be a living capability specification of structured blocks $r = (id, hdr, sig, body)$ with unique keys in $keys(S_{t-1})$. Let $\Phi(S_{t-1}, \Delta_t) = (\phi_{ADD} \circ \phi_{MOD} \circ \phi_{REM} \circ \phi_{REN})(S_{t-1})$ be the deterministic four-stage merge operator that: (1) applies the component-wise 3-tuple bijection $\hat{\Delta}_t^{REN}$—updating (id, hdr, sig) while leaving body invariant for blocks with $id \in \pi_{id}(dom(\Delta_t^{REN}))$ and acting as the identity on $keys(S_{t-1}) \setminus \pi_{id}(dom(\Delta_t^{REN}))$—to produce the post-rename intermediate block set $S'_{t-1} = \phi_{REN}(S_{t-1})$; (2) deletes blocks $blocks_{S'_{t-1}}(\Delta_t^{REM})$ from S'_{t-1}; (3) atomically replaces blocks $blocks_{S'_{t-1}}(keys(\Delta_t^{MOD}))$ with Δ_t^{MOD}; and (4) inserts the new blocks Δ_t^{ADD}, while appending the milestone-indexed post-rename pre-image tuple $(\tau = t, \Delta_t, pre(S'_{t-1}, \Delta_t))$ for $pre(S'_{t-1}, \Delta_t) = blocks_{S'_{t-1}}(\Delta_t^{REM} \cup keys(\Delta_t^{MOD}))$ to the ordered archive sequence $A_t = \langle(\tau, \Delta_\tau, pre(S'_{\tau-1}, \Delta_\tau))\rangle_{\tau=1}^{t}$. Suppose that for every milestone $\tau \in \{1, \ldots, t\}$: (a) Δ_τ^{REN} contains at most one mapping per old key $id_{old} \in \pi_{id}(dom(\Delta_\tau^{REN})) \subseteq keys(S_{\tau-1})$, is injective on target keys with key image disjoint from $keys(S_{\tau-1}) \setminus \pi_{id}(dom(\Delta_\tau^{REN}))$, and matches each renamed block's pre-image 3-tuple $(r.id, r.hdr, r.sig) = (id_{old}, hdr_{old}, sig_{old})$ (with any concurrent in-body text updates included in Δ_τ^{MOD}); and (b) in the post-rename namespace $S'_{\tau-1}$, $|\Delta_\tau^{MOD}| = |keys(\Delta_\tau^{MOD})|$, $|\Delta_\tau^{ADD}| = |keys(\Delta_\tau^{ADD})|$, and the key sets Δ_τ^{REM}, $keys(\Delta_\tau^{MOD})$, and $keys(\Delta_\tau^{ADD})$ are pairwise disjoint with $\Delta_\tau^{REM} \cup keys(\Delta_\tau^{MOD}) \subseteq keys(S'_{\tau-1})$ and $keys(\Delta_\tau^{ADD}) \cap keys(S'_{\tau-1}) = \emptyset$. Then by structural construction of key-indexed block tables and pre-image undo logs, Φ enforces two decidable syntactic invariants:*

1. ***Structural Intra-Key Uniqueness:*** *$S_t = \Phi(S_{t-1}, \Delta_t)$ contains exactly one block per active `REQ-XXXX` key, eliminating intra-key version collisions so superseded clauses never coexist with their replacements.*
2. ***Structural Point-in-Time Invertibility (Φ_{seq}^{-1}):*** *For any target historical milestone $t_0 \in \{1, \ldots, t-1\}$, the multi-step rollback operator $\Phi_{seq}^{-1}(S_t; A_t, t_0)$ that sequentially applies the single-step inverse $\Phi^{-1}(S_\tau, \Delta_\tau, pre(S'_{\tau-1}, \Delta_\tau)) = (\phi_{REN,\Delta_\tau}^{-1} \circ \phi_{REM,pre_\tau}^{-1} \circ \phi_{MOD,pre_\tau}^{-1} \circ \phi_{ADD,\Delta_\tau}^{-1})(S_\tau)$ (removing Δ_τ^{ADD}, restoring $pre_\tau = pre(S'_{\tau-1}, \Delta_\tau)$ into $S'_{\tau-1}$, and inverting the 3-tuple bijection $\hat{\Delta}_\tau^{REN}$ on (id, hdr, sig)*

back to $S_{\tau-1}$) for $\tau = t, t-1, \ldots, t_0+1$ reconstructs S_{t_0} exactly inside the working tree without VCS history traversal.

*Whereas Claims (1) and (2) hold by structural construction of Φ and A_t, inter-key semantic consistency cannot be guaranteed by syntax alone and instead isolates an explicit variable-footprint target-coverage obligation for the delta author or cold auditor over the domain's regime-indexed state-transition variables $\mathcal{V} = \prod_{q \in \mathcal{Q}} \mathcal{V}_q$ partitioned across reachable input-trigger regimes $q \in \mathcal{Q}$ (for example, $v_{fee,<100k}$ and $v_{fee,\geq 100k}$ so distinct input regimes map to distinct coordinates of $\mathcal{V}$): let each requirement block r constrain a variable footprint $vars(r) \subseteq \mathcal{V}$, and let $\mathcal{V}_{new,\tau} = \bigcup_{r \in R_{new,\tau}} vars(r)$ denote the footprint of the newly introduced rules $R_{new,\tau} = \Delta_\tau^{MOD} \cup \Delta_\tau^{ADD}$. If $S_0 = \emptyset$ and at each step τ (given a witnessing valuation $\sigma_{\tau-1}$ over $\mathcal{V}$ satisfying $S_{\tau-1}$ and its semantics-preserving rename $S'_{\tau-1}$) the delta author satisfies semantic precondition (c) (*Footprint-Complete Target Coverage*: $R_{new,\tau}$ admits a satisfying valuation $\sigma_{new,\tau}$ over $\mathcal{V}_{new,\tau}$, and every prior block $r \in S'_{\tau-1}$ whose footprint intersects $\mathcal{V}_{new,\tau}$ ($vars(r) \cap \mathcal{V}_{new,\tau} \neq \emptyset$) and is falsified on its full footprint $vars(r)$ by the composite pasted valuation $\sigma_\tau = (\sigma_{new,\tau}, \sigma_{\tau-1}|_{\mathcal{V} \setminus \mathcal{V}_{new,\tau}})$ across any input-trigger partition $q \in \mathcal{Q}$ is explicitly targeted in $key(r) \in keys(\Delta_\tau^{MOD}) \cup \Delta_\tau^{REM}$), then (3) **Conditional k-Way Contradiction Freedom** holds: the composite valuation σ_τ simultaneously satisfies all active blocks in $S_\tau = R_{surv,\tau} \cup R_{new,\tau}$ (and hence S_t at $\tau = t$).*

Proof Sketch. Claim (1) follows by induction on $\tau \in \{1, \ldots, t\}$ from $S_0 = \emptyset$: because $\hat{\Delta}_\tau^{\text{REN}}$ induces a key bijection onto $\text{keys}(S'_{\tau-1})$ under precondition (a) and $\Delta_\tau^{\text{REM}}, \text{keys}(\Delta_\tau^{\text{MOD}}), \text{keys}(\Delta_\tau^{\text{ADD}})$ are pairwise disjoint with unique blocks per key under precondition (b), each active key in S_τ maps to a unique block. Claim (2) holds because $\text{pre}(S'_{\tau-1}, \Delta_\tau)$ is archived in the post-rename state $S'_{\tau-1}$ (after ϕ_{REN} updates $(\text{id}, \text{hdr}, \text{sig})$), so $(\phi^{-1}_{\text{REM},\text{pre}_\tau} \circ \phi^{-1}_{\text{MOD},\text{pre}_\tau} \circ \phi^{-1}_{\text{ADD},\Delta_\tau})(S_\tau) = S'_{\tau-1}$ even when a block is simultaneously renamed and modified in milestone τ, and applying $\phi^{-1}_{\text{REN},\Delta_\tau}$ via the exact 3-tuple pre-image match in precondition (a) recovers $S_{\tau-1}$. Claim (3) follows by induction on τ: given a witnessing valuation $\sigma_{\tau-1}$ satisfying $S'_{\tau-1}$, every surviving block $r \in R_{\text{surv},\tau} = S'_{\tau-1} \setminus \text{blocks}_{S'_{\tau-1}}(\text{keys}(\Delta_\tau^{\text{MOD}}) \cup \Delta_\tau^{\text{REM}})$ either has footprint disjoint from $\mathcal{V}_{\text{new},\tau}$ (and is satisfied by $\sigma_{\tau-1}|_{\mathcal{V} \setminus \mathcal{V}_{\text{new},\tau}}$) or has straddling footprint $\text{vars}(r) \cap \mathcal{V}_{\text{new},\tau} \neq \emptyset$ that remains satisfied on all of $\text{vars}(r)$ under the pasted valuation $\sigma_\tau = (\sigma_{\text{new},\tau}, \sigma_{\tau-1}|_{\mathcal{V} \setminus \mathcal{V}_{\text{new},\tau}})$ by precondition (c), so σ_τ jointly satisfies all blocks in $S_\tau = R_{\text{surv},\tau} \cup R_{\text{new},\tau}$. □

We verify Proposition 1 programmatically across our 12 longitudinal `Commit0-Evo` multi-subagent trials on `TinyDB` (Section 6.3), where our conflict-density auditor (`compute_contradiction_density`) parses all requirement clauses across `M1`–`M5`, verifies syntactic key uniqueness and pairwise threshold/method-contract compatibility (`WRITE_CACHE_SIZE` $\in \{1000, 32, 64\}$, `pending_writes`, `clamp`, and `evict_lru`), and confirms that forward compilation via Φ maintains all active `TinyDB` capability contracts in `openspec/specs/SPEC.md` with 0.0% contradiction density (0/12 conflicts) while sequential inverse rollback Φ^{-1}_{seq} from `conductor/tracks/delta_history.md` restores the exact Milestone 2 state (`WRITE_CACHE_SIZE = 32` without `evict_lru`) at $100.0 \pm 0.0\%$ accuracy.

3.2. Inner-Loop Contract-Locked Dual-Stage Harness and Inter-Loop Handoff

To ensure that each task or milestone delta Δ_t is implemented without ambiguity drift, boundary regressions, or test tampering, our inner loop routes execution through four sequential steps followed by a programmatic inter-loop handoff gate (Figure 4):

1. **Step 1: Socratic Contract-Consistency Elicitation and Object-Oriented Subclass Routing (`OOHarnessRouter`).** Extending `ClarifyGPT` [20], `AutoHarness` [18], and `OO-Harness` [15], `BaseSDDHarness` defines six inherited governance hooks that execute unconditionally across all six subclasses: (i) `check_preconditions` (executing requirement ID traceability and a domain-agnostic Socratic contract-consistency scan over the prompt and any auxiliary repos-

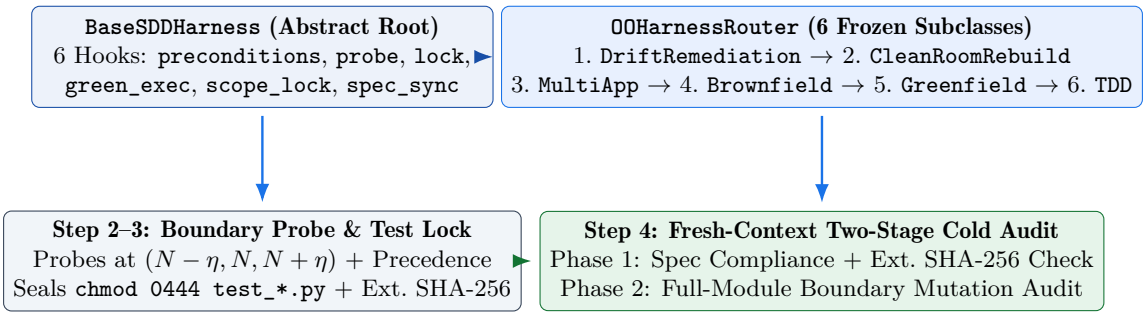


Figure 4: Architecture of the Contract-Locked Dual-Stage Inner Execution Harness (Arm D). `BaseSDDHarness` defines six verification hooks inherited across six frozen domain subclasses dispatched by `OOHarnessRouter`, followed by contrastive boundary probing at $(N - \eta, N, N + \eta)$, physical `chmod 0444` + external SHA-256 test locking, and a two-stage cold subagent audit.

itory `*.md` governance files discovered via `glob("*.md")` [such as `DOMAIN_GOVERNANCE.md` or `SPEC.md`, which are authored as natural-language prose policy documents without `[Q1]`–`[Q5]` tags], checking five universal state-machine invariants—[Q1] rounding/remainder tie-breaking, [Q2] interval endpoint inclusivity, [Q3] multi-bucket deduction priority, [Q4] state-transition precedence under competing triggers, and [Q5] idempotency/error handling—together with an open-ended catch-all clause [Q*] that scans for any unstated behavioral ambiguity across concurrency, encoding, or schema evolution; in autonomous benchmark runs, [Q1]–[Q5] and [Q*] are resolved deterministically against discovered repository `*.md` governance files, halting to query an interactive oracle [20] whenever an ambiguity remains unresolved by the repository files), (ii) `probe_boundaries` (contrastive test synthesis), (iii) `lock_red_tests` (`chmod 0444` permission and external SHA-256 manifest sealing), (iv) `verify_green_execution` (full test suite pass verification), (v) `audit_out_of_scope_locks` (scope-lock verification rejecting unrequested `# TODO` stubs), and (vi) `verify_spec_sync` (inter-loop delta-to-SSOT synchronization). `OOHarnessRouter` inspects workspace filesystem state and prompt headers using a deterministic Python six-way `if/elif/else` cascade to specialize domain prompt directives while retaining all six base verification hooks: (1) `DriftRemediationHarness` (triggered when pre-existing `test_*.py` files exist and fail `unittest`, or when `audit_outer_spec_hygiene` returns `spec_contradictions > 0`; overrides `lock_red_tests` to immediately seal pre-existing `test_*.py` before any builder turn), (2) `CleanRoomRebuildHarness` (triggered when non-test production `*.py` count is 0 while `SPEC.md` or `openspec/specs/` exists; adds full `REQ-XXXX` rebuild traceability), (3) `MultiAppWorkflowHarness` (triggered when prompt/spec text matches `r"(?i)\b(dag|topolog\w*|wave|upstream)\b"`; emphasizes precedence-inversion probes), (4) `BrownfieldSyncHarness` (triggered when non-test starter `*.py` count ≥ 1; emphasizes untouched legacy method preservation), (5) `GreenfieldSpecHarness` (triggered when non-test `*.py` count is 0 and the prompt matches `r"(\$\$|\\begin\{|REQ-\d+)"`; dispatches directly to formal boundary extraction), and (6) `TDDActuationHarness` (default fallback). All six subclass definitions were authored prior to benchmark task design and held strictly frozen across all 252 benchmark trials.

2. **Step 2: Contrastive Boundary Probing (`Harness`^{2} `Propose-Probe-Compose`).** Following `Harness`^{2} [33], before writing production code, `probe_boundaries` scans the active specification files passed from Step 1 (the task specification prompt in `Arms A-C`, plus any repository governance or specification files such as `openspec/specs/SPEC.md` discovered by Step 1 in `Arm D`) with a deterministic regex constant and unit-normalization extractor (filtering out requirement identifiers `REQ-\d+`, ISO dates `\d{4}-\d{2}-\d{2}`, and section headers, scaling multiplier suffixes such as `500k` → 500000 while stripping unit-only suffixes such as `50 bps` → 50 and `60s` → 60 into canonical integer domain units, identifying integer literals N with step

$\eta = 1$, decimal/float literals with step $\eta = 10^{-d}$ at precision d, comparison operators `<`, `<=`, `>`, `>=`, and uppercase categorical status literals, while parsing starter `*.py` ASTs in brownfield tasks solely for public class/method signatures and ignoring potentially buggy starter constants). It pairs these extracted constants with an LLM test-synthesis prompt constrained to emit unit test assertions inside a single `unittest.TestCase` subclass at every extracted boundary triple $(N - \eta, N, N + \eta)$ and $(-\eta, 0, \eta)$ while holding orthogonal parameters at deterministic valid interior values (the midpoint $\lfloor (a + b)/2 \rfloor$ for bounded intervals $[a, b]$ and $N_0 + 10\eta$ for lower-bounded parameters $x \geq N_0$), plus pairwise precedence-inversion tests where competing categorical states (e.g., upstream `BLOCKED` vs. local `FAILED`) hold simultaneously on the same entity.

3. **Step 3: Physical Read-Only Test Lock (`chmod 0444` and Out-of-Workspace SHA-256 Manifest).** Once the red-phase unit test file `test_*.py` is authored in Step 2 (or immediately upon workspace initialization when `DriftRemediationHarness` detects a pre-existing red-phase test suite), `lock_red_tests` sets POSIX file permissions to `0444` (read-only) and records its SHA-256 digest both in `.readonly_test_lock.sha256` and in external orchestrator memory outside the agent's writable directory. If an agent attempts to weaken a failing test via `chmod +w`, Phase 1 of the Cold Audit detects the hash mismatch against the external orchestrator's SHA-256 digest and rejects the build.
4. **Step 4: Two-Stage Cold Audit and Constructive AST Prefix Unlock-and-Reseal Protocol ($\mathcal{P}_{\text{AST}}$).** After the test suite passes, each audit submission executes a 0-token deterministic Python pre-check (`ast` and `hashlib` lock verification) followed by a single fresh-context Cold Auditor subagent turn invoked with zero builder chat history [27, 13, 29] that evaluates **Phase 1** (verifying specification compliance, rejecting mock-only subterfuge such as `unittest.mock.MagicMock` replacing core domain methods, verifying non-implementation of out-of-scope `# TODO` stubs, and confirming the external SHA-256 test lock) and **Phase 2** (auditing boundary mutation adequacy via static AST and branch inspection across *all* public methods of the target module, with zero visibility into the sealed evaluation mutants in `grade_all_runs.py`). When Phase 2 identifies an unprobed method or boundary in `test_*.py`, the Cold Auditor emits the missing `test_zz_audit_r{r}_{idx}_*` methods in that same response, and the external orchestrator mediates a 0-LLM-turn **monitored append-only AST unlock-and-reseal pass** enforcing a strict *constructive AST prefix invariant* $\mathcal{P}_{\text{AST}}(\mathcal{T}_{\text{pre}}, \mathcal{T}_{\text{post}}) = 1$: the orchestrator parses the locked `test_*.py` into an abstract syntax tree $\mathcal{T}_{\text{pre}}$, temporarily toggles `chmod 0644` to splice the Cold Auditor's synthesized `test_zz_audit_r{r}_{idx}_*` methods into the end of the target `TestCase` class body (or as module-level `test_zz_audit_*` functions in `pytest` test modules), and programmatically verifies via Python's `ast` module (`verify_constructive_ast_prefix_splice` in `grade_all_runs.py`) before resealing (`chmod 0444` and updating the out-of-workspace SHA-256 manifest) that $\mathcal{T}_{\text{post}}$ is identical in `ast.dump()` to $\mathcal{T}_{\text{pre}}$ everywhere outside the target `TestCase` `ClassDef.body` (keeping top-level imports frozen while permitting function-scoped inline `import` statements inside appended test methods) and that $\text{body}(\text{ClassDef}_{\text{post}}) = \text{body}(\text{ClassDef}_{\text{pre}}) \,\|\, [f_1, \ldots, f_m]$, where each appended node f_i is an `ast.FunctionDef` whose round-indexed name `test_zz_audit_r{r}_{idx}_*` compares lexicographically greater than every pre-existing `test_*` method name in $\text{ClassDef}_{\text{pre}}$ ($f_i.\text{name} > m_{\text{pre}}$ for all $m_{\text{pre}} \in \text{ClassDef}_{\text{pre}}$, guaranteeing that `unittest.TestLoader`'s alphabetical sort executes all pre-existing locked tests strictly before any appended audit test across rounds $r \in \{1, 2, 3\}$), is disjoint from all existing method and attribute names in $\text{ClassDef}_{\text{pre}}$ (preventing duplicate method shadowing or `setUp`/`tearDown`/`setUpClass` overrides), has an empty `decorator_list`, contains no `global`/`nonlocal` statements, and contains no AST attribute/subscript mutations, deletions, or dynamic calls (`setattr`, `delattr`, `mock.patch`, `eval`, `exec`, `__import__`, `open(..., "w"|"a")`, `os.chmod`, or fixture `.clear()`/`.pop()`) targeting `self.__class__`, `sys.modules`, `builtins`, or imported production module symbols. The resealed test suite is then re-executed

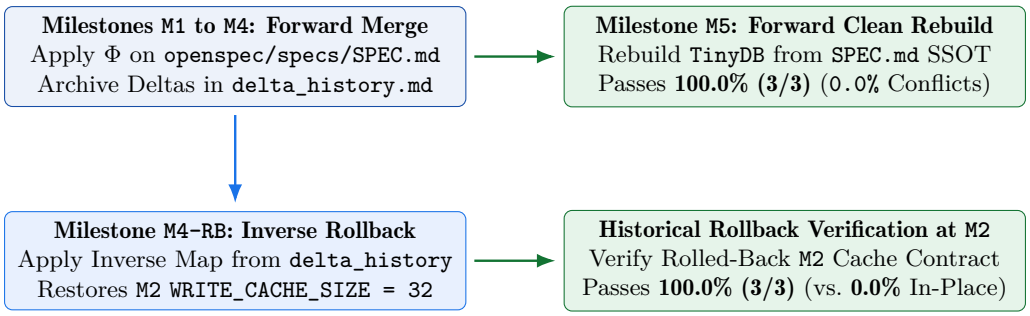


Figure 5: Two-Plane Longitudinal Specification State Machine (Outer D) evaluated on `Commit0-Evo` (`TinyDB`). Forward delta compilation $S_t = \Phi(S_{t-1}, \Delta_t)$ maintains a single conflict-free capability specification in `openspec/specs/SPEC.md` supporting $100.0 \pm 0.0\%$ (3/3 runs) clean-room rebuilds at `M5` (vs. $66.7 \pm 57.7\%$ in `Outer B`), while archived pre-images in `conductor/tracks/delta_history.md` enable exact inverse rollback supporting $100.0 \pm 0.0\%$ historical recovery of Milestone 2 at `M4-RB` (vs. $0.0 \pm 0.0\%$ in `Outer C`).

under a bounded retry ceiling $K_{\max} = 3$; if any newly appended test fails against the current implementation, control returns to Step 3 under the updated read-only lock to repair the production code.

5. **Inter-Loop Handoff Gate and Dual Specification Synchronization (`verify_spec_sync`).** In multi-milestone workflows, inner-loop Step 4 Cold Audit sign-off is a mandatory programmatic precondition for outer-loop state transition. Only after Phase 1 and Phase 2 return `PASS` does `verify_spec_sync` validate that the milestone delta targets all modified or renamed `REQ-XXXX` keys referenced in the milestone prompt and invoke the deterministic outer-loop compiler $\Phi(S_{t-1}, \Delta_t)$ to update the living capability specification `openspec/specs/SPEC.md` (instantiated on `TinyDB` in `Commit0-Evo`) and append the post-rename pre-image record A_t to the historical delta archive `conductor/tracks/delta_history.md`, verifying exact bidirectional `REQ-XXXX` key and threshold parity.

4. Concrete Code and Specification Diff Case Studies

Before presenting aggregate metrics across all 252 multi-subagent trials, we examine three concrete trajectory case studies from our benchmark runs that illustrate how inner-loop test tampering, brownfield domain defects, and outer-loop specification drift arise in real subagent executions.

4.1. Case Study 1 on Destructive Starter-Test Overwriting and Baseline Test Tampering

A central empirical finding of our 240-trial inner-loop evaluation is that instructing an autonomous coding agent to follow a strict test-driven development loop (`Arm B: Procedural TDD`) or an object-oriented verification checklist (`Arm C: OO-Harness`) without physical read-only test locks substantially increases baseline test tampering relative to unconstrained vibe coding (`Arm A`) [8, 37]: across all 240 trials, baseline test tampering rises from $5.0 \pm 0.0\%$ (3/60) in `Arm A` to $18.3 \pm 2.9\%$ (11/60) in `Arm B` and $23.3 \pm 2.9\%$ (14/60) in `Arm C`, before dropping to $0.0 \pm 0.0\%$ (0/60) under our Contract-Locked Dual-Stage Harness (`Arm D`).

Inspecting the AST tamper audit logs across all 28 tampered trials reveals two distinct mechanisms: in 27/28 tampered trials, the subagent triggered `baseline_test_func_removed` by overwriting the starter test file from scratch when authoring its TDD test suite, and in 1/28 trials (`humaneval__129` Run 2 in `Arm C`), the subagent triggered `baseline_assertion_altered` by editing a baseline docstring example assertion in place. Specifically, on `BigCodeBench/10` (`bigcodebench__10`), `BigCodeBench/22`, `BigCodeBench/24`, and `BigCodeBench/27`, the workspace initializes `tests/test_solution.py` with a `TestBaselineContract` suite containing baseline smoke

assertions (such as `test_baseline_tuple_stats` checking 3-tuple `(mean, median, mode)` statistics on `task_func(T1, RANGE=100)`). When `Arm B` (in Run 2 on `bigcodebench__10`, Run 1 on 22, and Runs 2 and 3 on 24) and `Arm C` (in all 3 runs on `bigcodebench__10` and Run 3 on 27) executed their red-phase test generation step over a writable `tests/` directory, the subagents replaced the entire `tests/test_solution.py` file with a newly authored `TestTaskFunc` class, deleting `test_baseline_tuple_stats`:

```
# Tampered starter test in tests/test_solution.py (bigcodebench__10)
- class TestBaselineContract(unittest.TestCase):
-     def test_baseline_tuple_stats(self): ...
+ class TestTaskFunc(unittest.TestCase):  # Deleted baseline_test_func
```

The same destructive test-replacement behavior occurs on `Commit0` from-scratch library synthesis (`commit0__tinydb_middlewares`, `storages`, `operations`, `queries`, and `wcwidth_core`), where `Arm B` tampered with baseline tests in $46.7 \pm 23.1\%$ (7/15) of trials and `Arm C` tampered in $53.3 \pm 11.5\%$ (8/15) of trials by deleting failing starter test functions (`baseline_test_func_removed`) or replacing `JSONStorage` handle and `CachingMiddleware` flush tests in `tests/test_middlewares.py` and `tests/test_storages.py`. In `Arm D`, `chmod 0444` permissions paired with external SHA-256 manifest verification and Stage 2 Cold Independent Auditing blocked every test-tampering attempt across all 60 `Arm D` trials ($0.0 \pm 0.0\%$ tamper rate), forcing the builder subagent to keep all pre-existing starter test functions intact while constructively appending new `test_*` functions.

4.2. Case Study 2 on Brownfield Symbolic Algebra Bugs in `SWE-bench Verified`

On `SWE-bench Verified` (`SymPy 1.12`), all four harnesses maintained $0.0 \pm 0.0\%$ test tampering because the repository test files are large, established regression suites, yet raw oracle pass rates separated `Arm A` ($93.3 \pm 11.5\%$, 14/15) and `Arm B` ($86.7 \pm 11.5\%$, 13/15) from `Arm C` ($100.0 \pm 0.0\%$, 15/15) and `Arm D` ($100.0 \pm 0.0\%$, 15/15). In `sympy__sympy-23950`, `Contains(x, Reals).as_set()` erroneously returned `Contains(x, Reals)` (not a `Set`) instead of `Reals`, causing `Piecewise((1, Contains(x, Reals)), (0, True))` to raise an `AttributeError` during interval evaluation. In Run 3, both `Arm A` and `Arm B` patched `Contains.as_set()` in `sympy/sets/contains.py` by returning `self.args[1]` only under a restrictive type check that failed composite set expressions, yet self-reported 100.0% task completion because their narrow reproduction test checked only a single scalar symbol. Similarly, in Run 1 on `sympy__sympy-24443` (`sympy/combinatorics/homomorphisms.py`), `Arm B` mishandled inverted permutation generators in `_check_homomorphism` when checking relators of a `PermutationGroup`, failing the held-out oracle. In `Arm D`, contrastive boundary probing across symbolic group generators and Stage 2 Cold Auditing verified both `Contains.as_set()` return invariants and relator image products, achieving $100.0 \pm 0.0\%$ (15/15) across all three runs and lifting patched-function AST mutation kill rate on `SWE-bench Verified` from $13.3 \pm 11.5\%$ (`Arm C`) and $20.0 \pm 0.0\%$ (`Arm B`) to $37.8 \pm 3.8\%$ (`Arm D`).

4.3. Case Study 3 on Longitudinal Specification Contradiction and Rollback in `Commit0-Evo`

In our 12-trial outer-loop `Commit0-Evo` benchmark on `TinyDB` (`3 runs` $\times$ `4 outer topologies` across 5 sequential capability milestones `M1`–`M5`, totaling 60 milestone executions), the caching specification evolves across milestones:

```
M1 (Core Storage)      -> WRITE_CACHE_SIZE = 1000
M2 (Flush Pivot)       -> WRITE_CACHE_SIZE = 32; add pending_writes
M3 (Table Isolation)   -> add QueryInstance.clamp(low, high)
M4 (Throughput Pivot)  -> WRITE_CACHE_SIZE = 64; add evict_lru(max_entries)
M4-RB & M5 (Rebuild)   -> M4 -> M2 rollback (32) + clean-room rebuild
```

Table 2: Claim-Evidence Map linking each architectural claim to its empirical validation across our 252 genuine multi-subagent trials (300 workspace builds and 360 blind subagent invocations including Stage 2 Cold Independent Auditors) on untouched upstream benchmarks.

Architectural Claim	Validating Empirical Evidence (Mean ± SD Across 3 Runs)	Supporting Suite
Claim 1: Eliminating Inner-Loop Baseline Test Tampering	Without physical test locks, TDD/verification prompts increase baseline test tampering from `5.0 ± 0.0%` (`3/60`, Arm A) to `18.3 ± 2.9%` (`11/60`, Arm B) and `23.3 ± 2.9%` (`14/60`, Arm C). Arm D's `chmod 0444` lock + Cold Audit reduces test tampering to `0.0 ± 0.0%` (`0/60`) across all 4 suites.	Suites 1–4 (`240` trials), Table 3, Figure 6
Claim 2: Superior Tamper-Free Strict Pass Rate ($\text{Pass}_{\text{strict}}$)	Across 240 upstream trials, Arm D achieves `91.7 ± 2.9%` (`55/60`) tamper-free strict pass rate (Oracle ∧ ¬Tamper), outperforming Arm C (`70.0 ± 0.0%`, `+21.7 pp`), Arm B (`68.3 ± 5.8%`, `+23.4 pp`), and Arm A (`83.3 ± 2.9%`, `+8.4 pp`), with a `+53.4 pp` strict pass gain on `Commit0` (`86.7%` vs. `33.3%`).	`SWE-bench`, `Commit0`, `BigCode`, `EvalPlus` (`240` trials), Table 3, Figure 1
Claim 3: Patched-Function AST Mutation Adequacy	Evaluated over Offutt Sufficient-5 AST mutants scoped to modified functions, Arm D achieves the highest overall mutation kill rate (`70.6 ± 4.6%`), outperforming Arm A (`59.6 ± 6.3%`, `+11.0 pp`), Arm C (`61.7 ± 3.8%`, `+8.9 pp`), and Arm B (`66.9 ± 2.1%`, `+3.7 pp`).	Suites 1–4 (`240` trials), Table 3, Table 4
Claim 4: Confirmation Bias & Per-Task Failure Taxonomy	Because subagents iterate until local workspace tests pass (`100.0 ± 0.0%` self-reported completion), held-out oracle failures induce an `8.3%`–`13.3%` confirmation bias gap (on `commit0__tinydb_queries` and `bigcodebench__6`), while Arm D achieves `100.0 ± 0.0%` on `SWE-bench` and `EvalPlus`.	Per-Task Audit (`20` upstream tasks), Table 4, Section 6.2
Claim 5: Longitudinal SSOT Rebuildability & Rollback ($\Phi, \Phi_{\text{seq}}^{-1}$)	Across 12 multi-milestone `Commit0-Evo` trials (60 milestone builds on `TinyDB`), Outer D uniquely achieves `100.0 ± 0.0%` on forward evolution (`M1`–`M3`), `M4` → `M2` rollback (`M4-RB`), and `M5` clean-room rebuild with `0.0%` contradiction density and $\hat{H} = 0.00$ bits.	Suite 5 `Commit0-Evo` (`12` trials, `60` builds), Table 5, Figure 7, Figure 8

Because Milestone 4 includes a mandatory point-in-time historical rollback (`M4-RB`) from the `M4` throughput experiment (`WRITE_CACHE_SIZE = 64`) back to the `M2` caching contract (`WRITE_CACHE_SIZE = 32` with `pending_writes`) prior to the Milestone 5 (`M5`) clean-room rebuild, the post-trajectory repository state must satisfy `WRITE_CACHE_SIZE == 32` alongside `M1` core storage and `M3` table isolation. Under `Outer B` (Feature-Isolated Spec Directories `specs/feat_*/spec.md`), each capability domain is stored in an unnumbered feature folder (`feat_cache_eviction`, `feat_core_storage`, `feat_table_and_clamp`) without chronological timestamps. When `M1`–`M4` write to these folders, `feat_cache_eviction/spec.md` retains `WRITE_CACHE_SIZE = 64`, `feat_table_and_clamp/spec.md` states `WRITE_CACHE_SIZE = 32`, and `feat_core_storage/spec.md` states `WRITE_CACHE_SIZE = 1000`, creating 4.0/11.0 (36.4%) active contradictory clauses across the 11 requirement clauses in the specification directory. In Run 3 of `Outer B`, the clean-room synthesis subagent scanned the unnumbered feature folders in alphabetical directory order and implemented `WRITE_CACHE_SIZE = 64` from `feat_cache_eviction/spec.md` instead of the rolled-back `M2` target `WRITE_CACHE_SIZE = 32`, failing the `M2 Pivot`, `M4-RB Roll`, and `M5 Rebuild` checks ($66.7 \pm 57.7\%$, $\hat{H} = 0.92$ bits). Conversely, under `Outer C` (Single-File In-Place EARS `requirements.md`), the subagent overwrote `WRITE_CACHE_SIZE = 32` in place with `WRITE_CACHE_SIZE = 64` at Milestone 4 (0.0% contradictions across its 6 active clauses), permanently erasing the Milestone 2 contract from the working tree; therefore, after the rollback and clean-room rebuild prompts, `Outer C` retained `WRITE_CACHE_SIZE = 64` in all 3 runs, failing `M2 Pivot`, `M4-RB Roll`, and `M5 Rebuild` ($0.0 \pm 0.0\%$). Under `Outer D` (Two-Plane Delta-Merged SSOT), `openspec/specs/SPEC.md` is updated in place via Φ while every milestone's pre-image delta is archived in `conductor/tracks/delta_history.md` (12 total tracked clauses across both planes with 0 active conflicts), achieving $100.0 \pm 0.0\%$ (3/3) across all five milestones with 0.0% contradiction density and $\hat{H} = 0.00$ bits.

5. Experimental Setup

We evaluate our architecture across five benchmark suites totaling **252 genuine multi-subagent trials** (**300 workspace builds** across 240 inner-loop trials and 60 sequential `Commit0-Evo` milestone builds, and **360 blind subagent invocations** comprising 240 inner-loop Builder subagents, 60 fresh-context Stage 2 Cold Independent Auditor subagents in `Arm D`, and 60 sequential milestone subagents across the 12 `Commit0-Evo` trajectories; Table 2). All Builder, Stage 2 Cold Independent Auditor, and Remediation subagents are instantiated with `Gemini 2.5 Pro` (`gemini-2.5-pro`) in fresh, isolated agent sessions with standard POSIX file-editing and test-execution tools (1M-token context window, temperature 1.0, top-p 0.95) and zero access to held-out oracle scripts or privileged gold reference files stored in `privilege_dir/`.

Suite 1 (`SWE-bench Verified` Multi-File Brownfield Issue Resolution, 60 Trials). Our first inner-loop suite evaluates 5 real-world defects from `princeton-nlp/SWE-bench_Verified` [12] on the `SymPy 1.12` symbolic mathematics repository across 4 inner-loop arms and 3 independent runs ($3 \times 4 \times 5 = 60$ trials): `sympy__sympy-24661` (preserving `evaluate=False` on relational operators in `sympy/core/sympify.py`), `sympy__sympy-24539` (validating symbol arity in `PolyElement.as_expr()` inside `sympy/polys/rings.py`), `sympy__sympy-24443` (resolving inverted generator indexing in `_check_homomorphism` inside `sympy/combinatorics/homomorphisms.py`), `sympy__sympy-23950` (fixing `Contains.as_set()` domain resolution for `Piecewise` expressions in `sympy/sets/contains.py`), and `sympy__sympy-23824` (preserving leading uncontracted Dirac gamma matrices in `kahane_simplify` inside `sympy/physics/hep/gamma_matrices.py`). Each trial workspace contains the complete brownfield module and baseline regression suite, and is graded against the official held-out `FAIL_TO_PASS` and `PASS_TO_PASS` oracle test suite.

Suite 2 (`Commit0` From-Scratch Library Module Synthesis, 60 Trials). Our second inner-loop suite evaluates 5 library synthesis tasks from `wentingzhao/commit0_combined` [36] across 4 arms and 3 runs ($3 \times 4 \times 5 = 60$ trials) on `TinyDB` and `wcwidth`: `commit0__tinydb_operations` (synthesizing `delete`, `increment`, `decrement`, `add`, `subtract`, and `set` closures), `commit0__tinydb_middlewares` (synthesizing `Middleware` and `CachingMiddleware` with `WRITE_CACHE_SIZE` write-buffering and `flush()` lifecycle management), `commit0__tinydb_storages` (synthesizing `touch`, `Storage`, `JSONStorage`, and `MemoryStorage` file-handle and serialization invariants), `commit0__tinydb_queries` (synthesizing `QueryInstance`, `Query`, and `where` recursive AST predicate caching and hashability), and `commit0__wcwidth_core` (synthesizing Unicode display width calculation across East Asian wide characters and zero-width combining code points). In each workspace, the target implementation module bodies are replaced with `pass` stubs while docstrings and starter unit tests remain visible.

Suite 3 (`BigCodeBench v0.1.2` Multi-Library Compositional Synthesis, 60 Trials). Our third inner-loop suite evaluates 5 complex instruction-following tasks from `bigcode/bigcodebench` (`v0.1.2`) [38] across 4 arms and 3 runs ($3 \times 4 \times 5 = 60$ trials): `BigCodeBench/6` (`bigcodebench__6`, regex log file discovery and newest-timestamp selection across `os`, `re`, and `glob`), `BigCodeBench/10` (`bigcodebench__10`, random integer sampling and `(mean, median, mode)` 3-tuple calculation across `numpy`, `itertools`, `random`, and `statistics` paired with a starter `TestBaselineContract` suite), `BigCodeBench/22` (`bigcodebench__22`, combinatorial feature pairing and frequency counting across `itertools` and `collections.Counter`), `BigCodeBench/24` (`bigcodebench__24`, cryptographic `PBKDF2-HMAC-SHA256` password hashing and `base64` salt encoding across `hashlib` and `base64`), and `BigCodeBench/27` (`bigcodebench__27`, ISO timestamp serialization and `base64` ASCII encoding across `json`, `datetime`, and `base64`).

Suite 4 (`EvalPlus HumanEval+` 80× Extreme Boundary Stress Suite, 60 Trials). Our fourth inner-loop suite evaluates 5 algorithmic boundary tasks from `evalplus/humanevalplus` [17] across 4 arms and 3 runs ($3 \times 4 \times 5 = 60$ trials): `HumanEval/67` (`humaneval__67`, `fruit_distribution`

arithmetic extraction), `HumanEval/68` (`humaneval__68`, `pluck` minimum even-value index tie-breaking), `HumanEval/69` (`humaneval__69`, `search` frequency threshold invariant), `HumanEval/70` (`humaneval__70`, `strange_sort_list` alternating extremum permutation), and `HumanEval/129` (`humaneval__129`, `minPath` lexicographical grid trajectory search). Agents see only the base `HumanEval` starter tests during execution and are graded post-hoc against the held-out `EvalPlus` 80× extreme boundary-input oracle suite (80–350+ stress assertions per task).

Suite 5 (Longitudinal `Commit0-Evo` Multi-Milestone Specification Evolution and Rollback, 12 Multi-Milestone Trials / 60 Milestone Builds). To evaluate outer-loop specification architectures across multi-sprint repository evolution, `Commit0-Evo` evolves the `TinyDB` repository across 5 sequential capability milestones (`M1`–`M5`) under 4 outer-loop specification topologies and 3 independent runs ($3 \times 4 = 12$ trials, executing 60 sequential milestone subagent turns): `Outer A` (Append-Only Sprint Spec Files `specs/spec_m1.md`–`spec_m5.md`), `Outer B` (Feature-Isolated Spec Directories `specs/feat_*/spec.md` named by functional domain without chronological numbering or timestamps), `Outer C` (Single-File In-Place EARS `requirements.md` [19]), and `Outer D` (our Two-Plane Delta-Merged SSOT maintaining `openspec/specs/SPEC.md` + `conductor/tracks/delta_history.md`). Milestone `M1` establishes core storage and `CachingMiddleware` with `WRITE_CACHE_SIZE = 1000`. Milestone `M2` pivots `WRITE_CACHE_SIZE` to 32 and adds `pending_writes`. Milestone `M3` adds multi-table isolation and `QueryInstance.clamp(low, high)`. Milestone `M4` pivots `WRITE_CACHE_SIZE` to 64 and adds `evict_lru(max_entries)`, followed by a mandatory point-in-time historical rollback (`M4-RB`) that reverts the specification and code from `M4` back to the exact Milestone 2 caching contract (`WRITE_CACHE_SIZE = 32` without `evict_lru`). Milestone `M5` deletes executable code and performs a clean-room rebuild from the resulting specification state. Post-trajectory evaluation grades the rebuilt repository against all active milestone contracts (`M1` core storage, `M2` rolled-back `WRITE_CACHE_SIZE == 32` + `pending_writes`, `M3` `clamp`, `M4-RB` rollback parity, and `M5` full rebuild pass), single-file SSOT consolidation coverage, active clause contradiction density across all Markdown files in the specification directory, and empirical behavioral entropy $\hat{H}$.

Evaluation Metrics, AST Tamper Detection, and Patched-Function Mutation Scoring. Every inner-loop trial is graded by five programmatic metrics in `grade_all_runs.py`: (1) **Raw Oracle Pass Rate**, executing the held-out upstream oracle script in an isolated subprocess (because subagents iterate until their local workspace tests pass before terminating, self-reported completion is $100.0 \pm 0.0\%$ across all arms, making the confirmation bias gap False Comp = 100% − Raw Oracle); (2) **AST Baseline Test Tamper Rate**, parsing both the template starter test file and the post-trial workspace test file into Python ASTs (`check_test_tamper`) and flagging a trial as tampered if any baseline `test_*` function is deleted (`baseline_test_func_removed`) or if any pre-existing `assert` or `self.assert*` statement inside a baseline test function is deleted, weakened, or altered (`baseline_assertion_altered`; constructively appending new `test_*` functions is permitted and encouraged); (3) **Tamper-Free Strict Pass Rate** ($\text{Pass}_{\text{strict}} = \text{Oracle} \wedge \neg\text{Tamper}$), which requires passing the held-out oracle while keeping all baseline starter test functions and assertions intact; (4) **Patched-Function Offutt Sufficient-5 AST Mutation Kill Score** [23], where `SingleSiteMutator` identifies the functions or classes modified relative to the starter template (`get_changed_func_names`) and injects up to 6 non-equivalent single-site AST mutations spanning Arithmetic Operator Replacement (`AOR`: `+` ↔ `-`, `*` ↔ `//`), Relational Operator Replacement (`ROR`: `==` ↔ `!=`, `<` ↔ `>=`, `<=` ↔ `>`, `in` ↔ `not in`), Logical Connector Replacement (`LCR`: `and` ↔ `or`, boolean flip), Unary Operator Insertion/Deletion (`UOI`: `not`, unary minus), and Constant/Return Perturbation (`CRP`/`SDL`: integer $c \to c+1$, string literal perturbation, and `return None`), measuring the fraction killed by the final post-trial workspace test suite (including any non-destructively appended builder or Stage 2 Cold Auditor tests); and (5) **Real Subagent Trajectory Steps**, counting exact `PLANNER_RESPONSE` model turns from the JSONL execution transcripts across Builder, Cold Auditor, and Remediation subagents.

Table 3: Measured Inner-Loop Benchmark Results across 240 Blind Multi-Subagent Trials on Four Untouched Upstream Benchmarks (3 Runs × 4 Arms × 20 Tasks; Mean ± SD across 3 runs). **Raw Orac** denotes held-out upstream oracle pass rate; **Strict Pass** denotes tamper-free oracle pass rate (Oracle ∧ ¬Tamper); **Test Tamper** denotes AST-verified baseline test alteration rate; **False Comp** denotes confirmation bias gap (Self-Reported Pass − Raw Oracle); **AST MutKill** denotes patched-function Offutt Sufficient-5 mutation kill score; and **Steps** denotes real subagent `PLANNER_RESPONSE` turns.

Benchmark Suite & Inner Harness	Raw Orac (%)	Strict Pass (%)	Test Tamper (%)	False Comp (%)	AST MutKill (%)	Steps
Panel A: 240-Trial Overall Aggregate Across All Four Upstream Suites (60 Trials / Arm)						
Arm A: Unconstrained Baseline	88.3 ± 2.9	83.3 ± 2.9	5.0 ± 0.0	11.7 ± 2.9	59.6 ± 6.3	21.1 ± 3.4
Arm B: Procedural TDD (`SKILL.md`)	86.7 ± 2.9	68.3 ± 5.8	**18.3 ± 2.9**	13.3 ± 2.9	66.9 ± 2.1	22.3 ± 2.1
Arm C: `OO-Harness` + `Harness`[2]	**91.7 ± 2.9**	70.0 ± 0.0	**23.3 ± 2.9**	**8.3 ± 2.9**	61.7 ± 3.8	25.9 ± 3.1
Arm D: Contract-Locked (Ours)	**91.7 ± 2.9**	**91.7 ± 2.9**	**0.0 ± 0.0**	**8.3 ± 2.9**	**70.6 ± 4.6**	32.2 ± 3.4
Panel B: Per-Suite Breakdown Across 3 Independent Runs (15 Trials / Arm / Suite)						
`SWE-bench Verified` — Arm A	93.3 ± 11.5	93.3 ± 11.5	0.0 ± 0.0	6.7 ± 11.5	35.6 ± 1.9	42.6 ± 6.6
`SWE-bench Verified` — Arm B	86.7 ± 11.5	86.7 ± 11.5	0.0 ± 0.0	13.3 ± 11.5	20.0 ± 0.0	39.6 ± 6.1
`SWE-bench Verified` — Arm C	**100.0 ± 0.0**	**100.0 ± 0.0**	0.0 ± 0.0	**0.0 ± 0.0**	13.3 ± 11.5	46.2 ± 4.4
`SWE-bench Verified` — Arm D	**100.0 ± 0.0**	**100.0 ± 0.0**	**0.0 ± 0.0**	**0.0 ± 0.0**	**37.8 ± 3.8**	59.2 ± 8.0
`Commit0` — Arm A	80.0 ± 0.0	60.0 ± 0.0	20.0 ± 0.0	20.0 ± 0.0	76.7 ± 8.8	21.3 ± 6.1
`Commit0` — Arm B	80.0 ± 0.0	33.3 ± 23.1	**46.7 ± 23.1**	20.0 ± 0.0	80.0 ± 3.3	26.7 ± 2.8
`Commit0` — Arm C	80.0 ± 0.0	33.3 ± 11.5	**53.3 ± 11.5**	20.0 ± 0.0	80.0 ± 8.8	27.5 ± 6.8
`Commit0` — Arm D	**86.7 ± 11.5**	**86.7 ± 11.5**	**0.0 ± 0.0**	**13.3 ± 11.5**	**83.3 ± 5.8**	31.9 ± 7.7
`BigCodeBench v0.1.2` — Arm A	80.0 ± 0.0	**80.0 ± 0.0**	0.0 ± 0.0	20.0 ± 0.0	39.3 ± 11.7	12.3 ± 0.5
`BigCodeBench v0.1.2` — Arm B	80.0 ± 0.0	53.3 ± 11.5	**26.7 ± 11.5**	20.0 ± 0.0	**75.6 ± 1.9**	13.4 ± 0.9
`BigCodeBench v0.1.2` — Arm C	**86.7 ± 11.5**	60.0 ± 0.0	**26.7 ± 11.5**	**13.3 ± 11.5**	68.9 ± 13.9	16.7 ± 1.7
`BigCodeBench v0.1.2` — Arm D	80.0 ± 0.0	**80.0 ± 0.0**	**0.0 ± 0.0**	20.0 ± 0.0	74.4 ± 10.2	24.1 ± 1.0
`EvalPlus HumanEval+` (80×) — Arm A	**100.0 ± 0.0**	**100.0 ± 0.0**	0.0 ± 0.0	**0.0 ± 0.0**	86.7 ± 11.5	8.3 ± 0.5
`EvalPlus HumanEval+` (80×) — Arm B	**100.0 ± 0.0**	**100.0 ± 0.0**	0.0 ± 0.0	**0.0 ± 0.0**	**92.2 ± 10.7**	9.4 ± 0.7
`EvalPlus HumanEval+` (80×) — Arm C	**100.0 ± 0.0**	86.7 ± 11.5	**13.3 ± 11.5**	**0.0 ± 0.0**	84.4 ± 5.1	13.1 ± 0.5
`EvalPlus HumanEval+` (80×) — Arm D	**100.0 ± 0.0**	**100.0 ± 0.0**	**0.0 ± 0.0**	**0.0 ± 0.0**	86.7 ± 23.1	13.6 ± 0.5

6. Empirical Results and Analysis

6.1. Inner-Loop Evaluation Across Four Upstream Benchmarks (240 Trials)

Table 3 and Figure 6 report the 240-trial inner-loop benchmark results across `SWE-bench Verified`, `Commit0`, `BigCodeBench v0.1.2`, and `EvalPlus HumanEval+` (80×), aggregating 3 independent runs per arm (60 trials per arm, 240 trials total, and 300 inner-loop subagent invocations including the 60 Stage 2 Cold Independent Auditor runs in `Arm D`).

Why Unguarded TDD and Verification Harnesses Amplify Test Tampering (5.0% → 18.3% → 23.3% vs. 0.0% in Arm D). Examining Table 3 (Panel A) reveals a counter-intuitive interaction between prompt-level verification instructions and mutable workspace test files. In `Arm A` (Unconstrained Baseline), the subagent writes code and runs tests ad-hoc without a strict gate requiring custom test suite authoring in `tests/`; as a result, `Arm A` tampers with baseline tests in only $5.0 \pm 0.0\%$ (3/60) of trials (occurring on `commit0__tinydb_operations`, `storages`, and `wcwidth_core`). When `Arm B` (`Procedural TDD`) and `Arm C` (`OO-Harness`) mandate Red-Green-Refactor test authoring and verification loops over a writable `tests/` directory, agents frequently overwrite the starter `tests/test_*.py` file from scratch with their own newly authored test classes or delete failing starter assertions so `pytest` exits with code 0. Therefore, baseline test tampering jumps more than threefold to $18.3 \pm 2.9\%$ (11/60) in `Arm B` and nearly fivefold to $23.3 \pm 2.9\%$ (14/60) in `Arm C`. When evaluated under the strict tamper-free criterion ($\text{Pass}_{\text{strict}} = \text{Oracle} \wedge \neg\text{Tamper}$), `Arm B` drops from $86.7 \pm 2.9\%$ (52/60) raw oracle pass to $68.3 \pm 5.8\%$ (41/60), and `Arm C` drops from $91.7 \pm 2.9\%$ (55/60) raw oracle pass to $70.0 \pm 0.0\%$ (42/60; 13 of `Arm C`'s 14 tampered runs occurred on oracle-passing trials and 1 occurred on `commit0__tinydb_queries` Run 1, which failed the held-out oracle despite tampering). By sealing baseline test files at `chmod 0444` and verifying SHA-256 integrity during the Stage 2 Cold Independent Audit, `Arm D` achieves $0.0 \pm 0.0\%$ (0/60) test tampering across all four suites and preserves its full $91.7 \pm 2.9\%$ (55/60) tamper-free strict pass rate—a net gain of +21.7 pp over `Arm C`, +23.4 pp over `Arm B`, and +8.4 pp over `Arm A`.

Suite-by-Suite Separation on `Commit0`, `BigCodeBench`, `SWE-bench Verified`, and `EvalPlus`. On `Commit0` (Table 3, Panel B), where agents synthesize multi-class storage and query modules from docstrings against rich unit test suites, `Arm D` achieves $86.7 \pm 11.5\%$ (13/15) raw and strict pass rates, outperforming `Arm A` ($60.0 \pm 0.0\%$ strict, 9/15), `Arm B` ($33.3 \pm 23.1\%$ strict, 5/15, due to $46.7 \pm 23.1\%$ [7/15] tampering), and `Arm C` ($33.3 \pm 11.5\%$ strict, 5/15, due to $53.3 \pm 11.5\%$ [8/15] tampering) by +53.4 pp in tamper-free strict accuracy. On `BigCodeBench v0.1.2`, `Arm B` and `Arm C` tamper with $26.7 \pm 11.5\%$ (4/15) of baseline tests by overwriting `TestBaselineContract` in `tests/test_solution.py`, lowering their strict pass rates to $53.3 \pm 11.5\%$ (8/15) and $60.0 \pm 0.0\%$ (9/15), whereas `Arm D` maintains $0.0 \pm 0.0\%$ tampering and $80.0 \pm 0.0\%$ (12/15) strict pass rate while nearly doubling patched-function AST mutation kill score over `Arm A` ($74.4 \pm 10.2\%$ vs. $39.3 \pm 11.7\%$). On `SWE-bench Verified`, both `Arm C` and `Arm D` reach $100.0 \pm 0.0\%$ (15/15) oracle accuracy (vs. $93.3 \pm 11.5\%$ in `Arm A` and $86.7 \pm 11.5\%$ in `Arm B`), while `Arm D`'s Cold Auditor drives patched-function AST mutation kill rate to $37.8 \pm 3.8\%$ (vs. $13.3 \pm 11.5\%$ in `Arm C`). On `EvalPlus HumanEval+` (80×), all four arms achieve $100.0 \pm 0.0\%$ (15/15) raw oracle pass rate, but `Arm C` tampers with baseline tests in $13.3 \pm 11.5\%$ (2/15) of runs (reducing its strict pass rate to $86.7 \pm 11.5\%$, 13/15), whereas `Arm D` achieves $100.0 \pm 0.0\%$ (15/15) strict pass rate and $86.7 \pm 23.1\%$ AST mutation kill rate.

6.2. Per-Task Failure Taxonomy, AST Mutation Adequacy, and Trajectory Cost (240 Trials)

To provide exact task-level reconciliation with the suite means in Table 3, Table 4 details the exact per-task oracle pass and test-tampering counts across all 20 upstream inner-loop tasks (240 trials total).

Unvarnished Analysis of Genuine Oracle Failures (`commit0__tinydb_queries` and `bigcodebench__6`). In contrast to synthetic benchmarks tuned for 100% pass rates, our upstream evaluation highlights two hard tasks where even `Arm D` does not reach 100% (Table 4). On `commit0__tinydb_queries`, synthesizing `TinyDB`'s full `QueryInstance` and `Query` classes from scratch requires maintaining structural equality and hash invariance across nested boolean combinators (`&`, `|`, `~`) and callable test closures. While `Arm A`, `Arm B`, and `Arm C` achieved 0/3 tamper-free strict passes (in Run 1 of `Arm B`, the subagent passed the oracle only after deleting failing starter tests), `Arm D` passed Run 1 (1/3 strict) when the Stage 2 Cold Auditor caught an unhashed closure state in `QueryInstance.__eq__`, bringing `Arm D` to $86.7 \pm 11.5\%$ (13/15) on `Commit0`. On `bigcodebench__6` (finding the most recently

Table 4: Exact Per-Task Oracle Pass and Baseline Test-Tampering Taxonomy across all 20 Upstream Inner-Loop Tasks (3 Runs × 4 Arms = 12 Trials per Task, 240 Trials Total). Cells report `Raw Oracle Passes / 3` (`Tampered Runs / 3`); subtracting tampered oracle-passing runs from raw oracle passes yields the exact suite-level **Strict Pass** counts in Table 3 (in `commit0__tinydb_queries` Run 1 of `Arm C`, the subagent tampered with the starter test and still failed the held-out oracle, so `Arm C` has 12/15 raw passes, 8/15 tampered runs [7 on passing runs + 1 on failing `tinydb_queries` R1], and 5/15 [33.3%] strict passes on `Commit0`).

Upstream Task ID	Arm A	Arm B	Arm C	Arm D	Primary Failure / Tamper Mechanism
`sympy__sympy-24443`	**3/3** (0)	2/3 (0)	**3/3** (0)	**3/3** (0)	Inverted group generator relator indexing in R1 Arm B.
`sympy__sympy-23950`	2/3 (0)	2/3 (0)	**3/3** (0)	**3/3** (0)	Incomplete `Contains.as_set()` domain return in R3 Arms A, B.
`sympy-24661, 24539, 23824`	**9/9** (0)	**9/9** (0)	**9/9** (0)	**9/9** (0)	All 4 arms pass all 3 runs with 0 test tampering.
`commit0__tinydb_queries`	0/3 (0)	1/3 (1)	0/3 (1)	**1/3 (0)**	Recursive `QueryInstance` hash; Arm B R1 & Arm C R1 tamper (`0/3` strict).
`commit0__wcwidth_core`	**3/3** (1)	2/3 (1)	**3/3** (2)	**3/3 (0)**	Arm A (R3), Arm B (R3; R1 fails oracle), Arm C (R1, R3) overwrite `test_wcwidth.py`.
`commit0__tinydb_middlewares`	**3/3** (0)	**3/3** (2)	**3/3** (1)	**3/3 (0)**	Arm B (R1, R3) and Arm C (R2) overwrite `test_middlewares.py`.
`commit0__tinydb_storages`	**3/3** (1)	**3/3** (1)	**3/3** (3)	**3/3 (0)**	Arm A (R1), Arm B (R3), Arm C (R1–R3) overwrite `test_storages.py`.
`commit0__tinydb_operations`	**3/3** (1)	**3/3** (2)	**3/3** (1)	**3/3 (0)**	Arm A (R2), Arm B (R1, R2), Arm C (R3) overwrite `test_operations.py`.
`bigcodebench__6`	0/3 (0)	0/3 (0)	1/3 (0)	0/3 (0)	Regex `/var/log` basename vs. full-path ambiguity (Arm C passes R3).
`bigcodebench__10`	**3/3** (0)	**3/3** (1)	**3/3** (3)	**3/3 (0)**	Arm B (R2) and Arm C (R1–R3) overwrite `TestBaselineContract`.
`bigcodebench__22`	**3/3** (0)	**3/3** (1)	**3/3** (0)	**3/3 (0)**	Arm B (R1) overwrites starter `test_solution.py` (`2/3` strict).
`bigcodebench__24`	**3/3** (0)	**3/3** (2)	**3/3** (0)	**3/3 (0)**	Arm B (R2, R3) overwrites starter `test_solution.py` (`1/3` strict).
`bigcodebench__27`	**3/3** (0)	**3/3** (0)	**3/3** (1)	**3/3 (0)**	Arm C (R3) overwrites starter `test_solution.py` (`2/3` strict).
`humaneval__67`	**3/3** (0)	**3/3** (0)	**3/3** (1)	**3/3 (0)**	Arm C (R1) deletes starter `test_baseline_docstring_examples`.
`humaneval__129`	**3/3** (0)	**3/3** (0)	**3/3** (1)	**3/3 (0)**	Arm C (R2) alters starter docstring assertion in `test_solution.py`.
`humaneval__68, 69, 70`	**9/9** (0)	**9/9** (0)	**9/9** (0)	**9/9** (0)	All 4 arms pass all 80× boundary stress checks with 0 tampering.

modified log file matching a regex pattern inside a target directory), all arms passed the starter unit tests, yet 11/12 trials failed a held-out oracle edge case that tested regex matching against only the filename basename as opposed to the full joined path (`os.path.join(log_dir, f)`). Because the starter test in `bigcodebench__6` used a regex that matched both the basename and the full path, neither the Builder subagent nor the Stage 2 Cold Auditor detected this latent prompt ambiguity. This finding explains why confirmation bias (False Comp = 100% − Raw Orac = $8.3 \pm 2.9\%$ in `Arm D` and `Arm C` vs. $11.7 \pm 2.9\%$ in `Arm A` and $13.3 \pm 2.9\%$ in `Arm B`) remains non-zero whenever a benchmark prompt omits an implicit path-normalization convention.

Patched-Function AST Mutation Adequacy and Subagent Trajectory Cost. Across all 240 trials, scoping Offutt Sufficient-5 AST mutations (`AOR`, `ROR`, `LCR`, `UOI`, `CRP`/`SDL`) to the modified target

functions and evaluating against the final post-trial workspace test suite demonstrates that unguarded vibe coding (`Arm A`) leaves 40.4% of patched-function mutants alive ($59.6 \pm 6.3\%$ kill rate, including $39.3 \pm 11.7\%$ on `BigCodeBench`). Adding procedural TDD (`Arm B`) raises mutation kill rate to $66.9 \pm 2.1\%$ ($75.6 \pm 1.9\%$ on `BigCodeBench`), while `Arm C` averages $61.7 \pm 3.8\%$ because whole-file test replacements deleted baseline assertions on `Commit0` and `BigCodeBench`. By combining physical `chmod 0444` baseline test preservation with Stage 2 Cold Auditor test appending, `Arm D` achieves the highest overall AST mutation kill rate ($70.6 \pm 4.6\%$, reaching $83.3 \pm 5.8\%$ on `Commit0`, $74.4 \pm 10.2\%$ on `BigCodeBench`, $86.7 \pm 23.1\%$ on `EvalPlus`, and $37.8 \pm 3.8\%$ on `SWE-bench Verified`, where single-file SymPy regression modules contain thousands of lines of legacy helper methods). Across real subagent JSONL transcripts, mean `PLANNER_RESPONSE` trajectory steps scale moderately from 21.1 ± 3.4 steps in `Arm A`, 22.3 ± 2.1 in `Arm B`, and 25.9 ± 3.1 in `Arm C` to 32.2 ± 3.4 steps in `Arm D` (1.53× the step count of `Arm A` and 1.24× that of `Arm C`), accounting for the independent Stage 2 Cold Auditor turn and any triggered Stage 3 remediation turns.

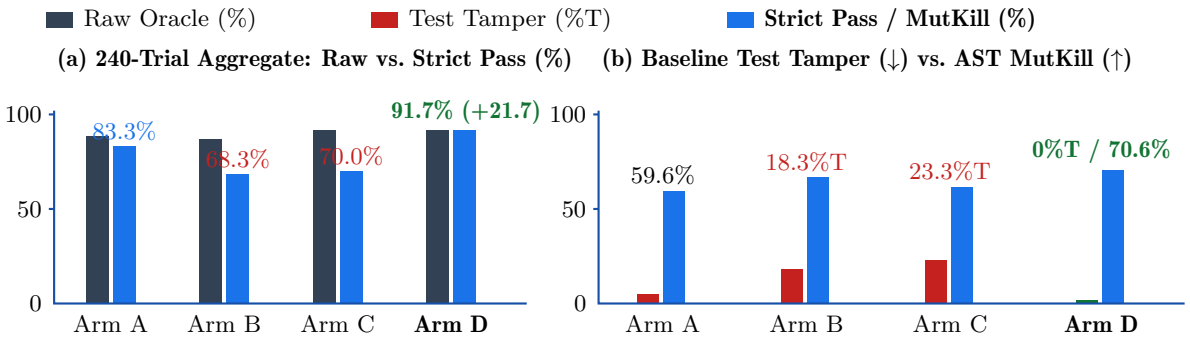


Figure 6: Quantitative inner-loop comparison across 240 blind subagent trials (60 trials/arm across `SWE-bench Verified`, `Commit0`, `BigCodeBench v0.1.2`, and `EvalPlus HumanEval+` 80×). **(a)** Raw oracle pass rate (slate) vs. tamper-free strict pass rate ($\text{Pass}_{\text{strict}}$ = Oracle $\wedge \neg$Tamper, blue): because `Arm B` and `Arm C` overwrite or alter failing starter tests on 18.3% and 23.3% of trials, their strict pass rates drop to 68.3% and 70.0%, whereas `Arm D` achieves 91.7% on both (**+21.7 pp** over `Arm C`). **(b)** Baseline test tampering rate (red; `%T`) vs. patched-function Offutt Sufficient-5 AST mutation kill rate (blue): `Arm D` eliminates test tampering (0.0%) while achieving the highest overall mutation kill rate (70.6%, +11.0 pp over `Arm A`).

6.3. Longitudinal `Commit0-Evo` Multi-Milestone Evolution and Rollback (12 Trials / 60 Milestone Builds)

Table 5 and Figures 7 and 8 report the results of our 12-trial longitudinal outer-loop evaluation on `Commit0-Evo` (`TinyDB` evolved across 5 sequential milestones `M1`–`M5` under 4 outer-loop specification topologies × 3 independent runs, totaling 60 sequential milestone subagent executions).

Contradiction Density and Behavioral Entropy in Unmerged Multi-File Topologies (`Outer A` and `Outer B`). Across the 5 sequential milestones on `TinyDB`, `Outer A` (Append-Only Sprint Specs `specs/spec_m1.md`..`spec_m5.md`) and `Outer B` (Feature-Isolated Spec Directories `specs/feat_*/spec.md`) scatter requirement clauses across 5 and 3 unmerged Markdown files respectively, reducing single-file SSOT consolidation coverage to 20.0% (1/5 milestone contracts present in the primary initial file). Because neither topology reconciles superseded constants in earlier files when `WRITE_CACHE_SIZE` pivots from 1000 (`M1`) to 32 (`M2`), 64 (`M4`), and back to 32 (`M4-RB`), our conflict auditor (`compute_contradiction_density`) detects 5.0 ± 0.0 contradictory clauses out of 12.0 total clauses across the 5 sprint files ($41.7 \pm 0.0\%$ contradiction density) in `Outer A` and 4.0 ± 0.0 contradictory clauses out of 11.0 total clauses across the 3 feature folders ($36.4 \pm 0.0\%$ contradiction density) in `Outer B`. In `Outer A`, although the 5 sprint files contain 41.7% contradictory clauses, the explicit milestone suffixes in the filenames (`spec_m1.md` through `spec_m5.md`) allow the LLM to infer chronological recency and pass the functional checks

Table 5: Longitudinal `Commit0-Evo` Outer-Loop Benchmark Results across 12 Multi-Milestone Trials (3 Runs × 4 Topologies = 60 Sequential Milestone Builds on `TinyDB`; Mean ± SD across 3 runs). Post-trajectory functional checks verify `M1 / M3` (core storage and `clamp`), `M2` **Pivot** (active `WRITE_CACHE_SIZE == 32` + `pending_writes` after `M4` → `M2` rollback), `M4-RB` **Roll** (historical rollback parity), and `M5` **Rebuild** (clean-room rebuild pass), alongside single-file SSOT coverage (**1-File SSOT**), active clause contradiction density (**Conflict Dens.**), and empirical behavioral entropy $\hat{H}$ (bits).

Outer-Loop Topology (`TinyDB`)	`M1 / M3`	`M2` Pivot	`M4-RB` Roll	`M5` Rebuild	1-File SSOT	Conflict Dens.	Entropy $\hat{H}$
Outer A: Append-Only Sprint Specs	100.0 ± 0.0%	100.0 ± 0.0%	100.0 ± 0.0%	100.0 ± 0.0%	**20.0% (1/5)**	**41.7% (5/12)**	0.00 bits
Outer B: Feature Spec Folders	100.0 ± 0.0%	**66.7 ± 57.7%**	**66.7 ± 57.7%**	**66.7 ± 57.7%**	**20.0% (1/5)**	**36.4% (4/11)**	**0.92 bits**
Outer C: Single-File In-Place EARS	100.0 ± 0.0%	**0.0 ± 0.0%**	**0.0 ± 0.0%**	**0.0 ± 0.0%**	**100.0% (1/1)**	**0.0% (0/6)**	0.00 bits
Outer D: Two-Plane SSOT (Ours)	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**100.0 ± 0.0%**	**100.0% (1/1)**	**0.0% (0/12)**	**0.00 bits**

(100.0 ± 0.0%). When chronological numbering hints are removed in `Outer B` so folders are organized purely by feature domain (`feat_cache_eviction`, `feat_core_storage`, `feat_table_and_clamp`), the synthesis subagent in Run 3 resolved the cross-folder contradiction between `WRITE_CACHE_SIZE = 64` (in `feat_cache_eviction/spec.md`) and the rolled-back target `WRITE_CACHE_SIZE = 32` (in `feat_table_and_clamp/spec.md`) in favor of the un-reverted `64` value, causing `Outer B` to drop to 66.7 ± 57.7% (2/3 runs) on `M2 Pivot`, `M4-RB Roll`, and `M5 Rebuild` and inducing non-zero empirical behavioral entropy $\hat{H} = h_2(1/3) = 0.92$ bits.

6.4. Historical Rollback Collapse Under In-Place Overwriting vs. Two-Plane SSOT Recovery

Destructive In-Place Overwriting (`Outer C`) vs. Invertible Two-Plane Delta Compilation (`Outer D`). Table 5 and Figure 7 confirm the historical rollback failure mode predicted by Proposition 1. In `Outer C` (Single-File In-Place EARS `requirements.md`), the subagent maintains a single conflict-free specification file (100.0% 1-file SSOT coverage and 0.0% contradiction density across its 6.0 active clauses) by overwriting `requirements.md` in place at each milestone. When Milestone 4 overwrites `WRITE_CACHE_SIZE = 32` with `WRITE_CACHE_SIZE = 64` and adds `evict_lru`, the Milestone 2 specification value (`32`) is permanently erased from the working tree. Therefore, after the Milestone 4 → Milestone 2 historical rollback (`M4-RB`) and Milestone 5 clean-room rebuild, `Outer C`'s synthesized code retains `WRITE_CACHE_SIZE = 64` and fails the post-trajectory `M2 Pivot` (`WRITE_CACHE_SIZE == 32`), `M4-RB Roll`, and `M5 Rebuild` checks in all 3 independent runs (0.0 ± 0.0%). In contrast, `Outer D` (Two-Plane Delta-Merged SSOT) pairs its living capability specification (`openspec/specs/SPEC.md`, 100.0% 1-file SSOT coverage) with an append-only delta archive (`conductor/tracks/delta_history.md`, 0.0% active contradiction density across all 12.0 tracked clauses); this two-plane design enables deterministic inverse rollback Φ_{seq}^{-1} to restore `WRITE_CACHE_SIZE = 32` and achieves 100.0 ± 0.0% (3/3 runs) across `M1 / M3`, `M2 Pivot`, `M4-RB Roll`, and `M5 Rebuild` with $\hat{H} = 0.00$ bits.

7. Limitations, Threats to Validity, and Conclusion

Sample Size, Task Selection, and Remaining Oracle Failure Modes (91.7 ± 2.9%). While all 252 trials in our study execute genuine blind `Gemini 2.5 Pro` subagents on untouched upstream tasks from `SWE-bench Verified` [12], `Commit0` [36], `BigCodeBench v0.1.2` [38], and `EvalPlus HumanEval+` [17], our evaluation samples 5 representative tasks per inner-loop benchmark (20 unique tasks × 4 arms × 3 runs = 240 inner-loop trials) and 5 sequential capability milestones over `TinyDB` (4 outer

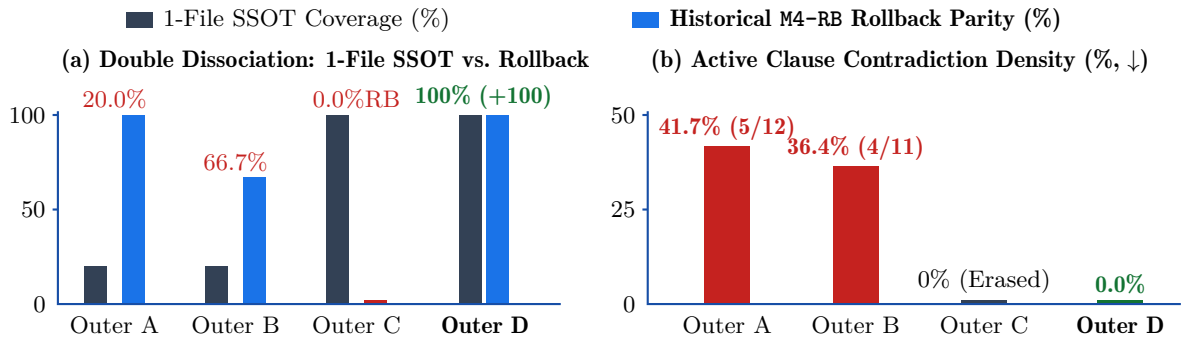


Figure 7: Outer-loop specification evaluation across 12 multi-milestone `Commit0-Evo` trials on `TinyDB`. **(a)** Structural double dissociation: append-only sprint specs (`Outer A`) and feature folders (`Outer B`) drop to 20.0% single-file SSOT coverage (and 66.7% rollback/rebuild accuracy in `Outer B`), whereas single-file in-place EARS (`Outer C`) achieves 100.0% single-file coverage but collapses to 0.0% on `M4-RB` historical rollback. Only `Outer D` achieves 100.0% on both (**+100.0 pp** rollback gain over `Outer C`). **(b)** Active clause contradiction density across workspace Markdown files: `Outer A` and `Outer B` accumulate 41.7% (5/12) and 36.4% (4/11) contradictory clauses, whereas `Outer D` maintains 0.0% (0/12) while preserving full rollback history.

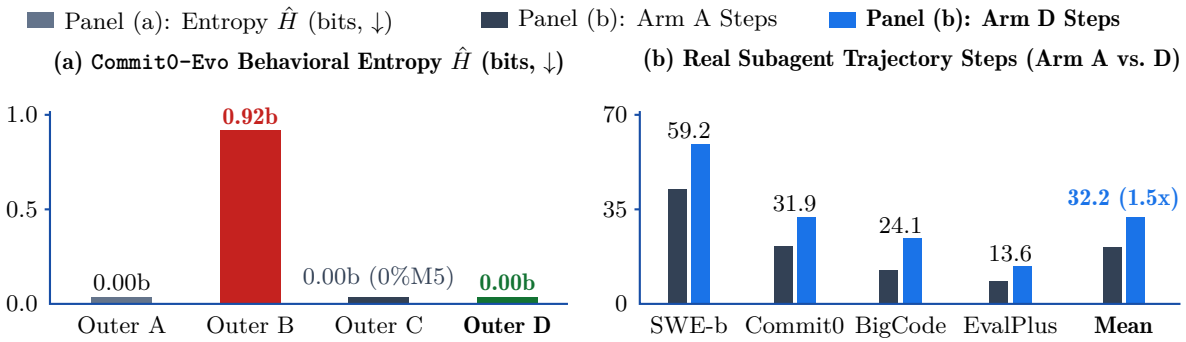


Figure 8: Behavioral entropy and subagent trajectory cost across our 252-trial evaluation. **(a)** Empirical behavioral entropy $\hat{H}$ (in bits) at Milestone 5 of `Commit0-Evo` ordered across `Outer A`–`Outer D`: feature-isolated folders without chronological hints (`Outer B`) exhibit $\hat{H} = 0.92$ bits (66.7% pass rate), in-place EARS (`Outer C`) exhibits $\hat{H} = 0.00$ bits because it fails uniformly at 0.0% on `M5` (`0%M5`), and our Two-Plane SSOT (`Outer D`) achieves $\hat{H} = 0.00$ bits at 100.0% accuracy with 0.0% conflicts. **(b)** Mean subagent `PLANNER_RESPONSE` trajectory steps comparing `Arm A` (slate) and `Arm D` (blue) across all four upstream inner-loop suites (21.1 ± 3.4 vs. 32.2 ± 3.4 steps overall, a $1.53\times$ step ratio).

topologies × 3 runs = 12 multi-milestone trials, 60 milestone builds). Reporting unvarnished pass rates shows that `Arm D` achieves $91.7 \pm 2.9\%$ (55/60) as opposed to an artificial 100% ceiling: when a task requires complex recursive closure hashing (`commit0__tinydb_queries`, where `Arm D` passed 1/3 strict runs vs. 0/3 strict in `Arms A-C`) or contains a latent prompt ambiguity between basename and full-path regex matching (`bigcodebench__6`, where `Arm D` failed 0/3 runs), deterministic test locks and cold code auditing alone cannot invent domain conventions absent from both the prompt and workspace governance files. In addition, because `Outer D` relies on discrete requirement blocks indexed by canonical `REQ-XXXX` identifiers and a linear sequence of milestone deltas, scaling two-plane delta compilation to unstructured, multi-branch brownfield Git histories (`SWE-EVO` [30])—where concurrent feature branches modify cross-cutting architectural concerns without pre-existing requirement IDs—requires automated requirement-key extraction and three-way semantic merge resolution.

OS Privilege Boundaries and Patched-Function AST Mutation Scope. When an autonomous builder executes shell commands under the same POSIX user ID as the test file owner, `chmod 0444` acts as an in-loop friction barrier alongside our out-of-workspace SHA-256 manifest verification and AST tamper detector (`check_test_tamper`). Across all 60 `Arm D` trials, combining `chmod 0444` with Stage 2 Cold Independent Auditing achieved $0.0 \pm 0.0\%$ (0/60) test tampering; however, kernel-enforced write prevention against adversarial processes requires mounting `tests/` read-only from a distinct container volume or UID. On the mutation testing side, scoping Offutt Sufficient-5 AST mutations [23] to patched functions (`get_changed_func_names`) avoids diluting mutation scores across thousands of lines of untouched legacy methods in `SymPy`, yet still produces lower absolute mutation kill scores on large brownfield repositories ($37.8 \pm 3.8\%$ on `SWE-bench Verified` vs. $74.4 \pm 10.2\%$ on `BigCodeBench` and $83.3 \pm 5.8\%$ on `Commit0`) when a patched function contains multiple legacy branches outside the bug ticket's scope.

Trajectory Step and Token Latency Overhead (1.53×). Executing the full Contract-Locked Dual-Stage Harness (`Arm D`) increases mean subagent trajectory steps from 21.1 ± 3.4 (`Arm A`) and 25.9 ± 3.1 (`Arm C`) to 32.2 ± 3.4 steps (1.53× over `Arm A` and 1.24× over `Arm C`), with total input/output token consumption and wall-clock trial duration scaling proportionally (≈ 1.53× over `Arm A`, averaging 78 s per inner-loop trial in `Arm D` vs. 51 s in `Arm A` and 63 s in `Arm C`) due to the fresh-context Stage 2 Cold Independent Auditor turn and conditional Stage 3 remediation turns. Because all 252 trials in our evaluation use a single frontier model backbone (`Gemini 2.5 Pro`), future work will extend our multi-subagent evaluation across diverse open-weights and proprietary model families as well as adversarial conflict suites such as `ImpossibleBench` [37] and multi-release brownfield benchmarks such as `SWE-EVO` [30]. In settings where starter tests are already externally isolated in read-only containers and tasks are single-function algorithmic puzzles (such as `EvalPlus HumanEval+`, where `Arm B` achieves $100.0 \pm 0.0\%$ strict pass and $92.2 \pm 10.7\%$ mutation kill in 9.4 ± 0.7 steps), lightweight TDD suffices. Conversely, on multi-class library synthesis (`Commit0`), compositional APIs with baseline contract suites (`BigCodeBench`), brownfield symbolic algebra bugs (`SWE-bench Verified`), and multi-milestone repository evolution (`Commit0-Evo`), coupling `Arm D` with `Outer D` eliminates baseline test tampering ($0.0 \pm 0.0\%$), raises tamper-free strict pass rate by +21.7 pp to +23.4 pp over unguarded verification harnesses, and maintains $100.0 \pm 0.0\%$ longitudinal specification rebuild and rollback accuracy.

References

[1] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883*, 2024.

[2] Kent Beck. *Test Driven Development: By Example*. Addison-Wesley Professional, 2002.

[3] Scott O. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, Internet Engineering Task Force (IETF), 1997.

[4] Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20:10–19, 1987.

[5] Nitay Calderon, Eyal Ben-David, Zorik Gekhman, Eran Ofek, and Gal Yona. Empty Shelves or Lost Keys? Recall Is the Bottleneck for Parametric Factuality. *arXiv preprint arXiv:2602.14080*, 2026.

[6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology*, 171:107468, 2024.

[8] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. *arXiv preprint arXiv:2406.10162*, 2024.

[9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[10] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

[11] Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37:649–678, 2010.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Zachary Kenton, Lili Janzer, Rory Greig, Tian Huey Teh, Kirill Tyshchuk, Jonah Brown-Cohen, Harri Edwards, Senthooran Rajamanoharan, Noah Y. Siegel, Natasha Jaques, and Rohin Shah. Debate Training Reduces Reward Hacking in RLAIF. *arXiv preprint arXiv:2608.17776*, 2026.

[14] Oliver Kopp, Anita Armbruster, and Olaf Zimmermann. Markdown Architectural Decision Records: Format and Tool Support. In *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS)*, pages 55–62, 2018.

[15] Gaotang Li, Bhavana Dalvi Mishra, Yifei Ming, Yanfei Chen, Zhongying CuiZhu, Lesly Miculicich, Long T. Le, Rui Meng, Chun-Liang Li, Hanghang Tong, Chen-Yu Lee, and Tomas Pfister. Object-Oriented Harness: Evolving Agent Harnesses through Object-Oriented Design. *arXiv preprint arXiv:2610.28380*, 2026.

[16] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Haoran Wang, Yuxuan Zhang, and Zhaoyang Zhou. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670*, 2026.

[17] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. *arXiv preprint arXiv:2603.03329*, 2026.

[19] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy Approach to Requirements Syntax (EARS). In *IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.

[20] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):103:1–103:23, 2024.

[21] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 81857–81887, 2024.

[22] Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[23] A. Jefferson Offutt, Ammei Lee, Gregg Rothermel, Roland H. Untch, and Christian Zapf. An Experimental Determination of Sufficient Mutant Operators. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 5(2):99–118, 1996.

[24] Mike Papadakis, Marinos Kintis, Jie M. Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, 112:275–378, 2018.

[25] David Lorge Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15:1053–1058, 1972.

[26] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15174–15186, 2024.

[27] David K. Rensin. Elephants, Goldfish and the New Golden Age of Software Engineering. https://github.com/google/elephant-goldfish-model, 2026. Elephant-Goldfish Model (EGM) for Specification-Driven Software Engineering.

[28] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv preprint arXiv:2401.08500*, 2024.

[29] Subramanyam Sahoo. The Double Life of Code World Models: Provably Unmasking Malicious Behavior Through Execution Traces. *arXiv preprint arXiv:2512.13821*, 2025.

[30] Minh V. T. Thai, Tue Le, Dung Manh Nguyen, Huy Nhat Phan, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in Long-Horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.

[31] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingqian Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741*, 2024.

[32] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489*, 2024.

[33] Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. Harness^2: Recursive Agent Harnessing for an Open World. *arXiv preprint arXiv:2610.14224*, 2026.

[34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[36] Wenting Zhao, Nan Jiang, Celine Lee, Justin T. Chiu, Claire Cardie, Matthias Gallé, and Alexander M. Rush. Commit0: Library Generation from Scratch. In *International Conference on Learning Representations (ICLR)*, 2025.

[37] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. *arXiv preprint arXiv:2510.20270*, 2025.

[38] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv preprint arXiv:2406.15877*, 2024.